UECM3034

NUMERICAL METHODS

Complete Course Notes

Topics 1–8

June 2026 Trimester

Definitions, theorems, algorithms and worked examples

CONTENTS

Chapter 1

MATHEMATICAL PRELIMINARIES

1.1 Introduction to Numerical Methods

Numerical methods are systematic procedures for obtaining approximate solutions to mathematical problems using a computer or calculator. They are especially useful when an exact analytic solution does not exist, is difficult to obtain, or is too complicated to be useful in practice.

A numerical computation usually contains the following:

(i) a mathematical model or equation;

(ii) an algorithm consisting of a finite sequence of steps;

(iii) an approximation produced by the algorithm; and

(iv) an assessment of the error and accuracy of that approximation.

Unlike symbolic mathematics, numerical mathematics must also account for the fact that computers store only finitely many digits. Therefore, even an algebraically correct formula may give a poor numerical answer if it is evaluated carelessly.

1.2 Types of Errors

Definition. (Basic errors) Let x be the true value and let $\widehat{x}$ be an approximation to x.

(a) The *error* is $e = x - \widehat{x}$.

(b) The *absolute error* is $|e| = |x - \widehat{x}|$.

(c) The *relative error* is $\varepsilon_r = \frac{x-\widehat{x}}{x}, \qquad x \neq 0$.

(d) The *percentage relative error* is $100\varepsilon_r\% = \frac{x-\widehat{x}}{x} \times 100\%$.

When only the magnitude matters, we consider the absolute relative error $\dfrac{|x - \widehat{x}|}{|x|}$.

Definition. (Round-off error) A *round-off error* occurs because a computer or calculator represents a real number using only finitely many digits. Arithmetic performed with these finite representations may differ from exact real-number arithmetic.

Definition. (Truncation error) A *truncation error* results from replacing an exact mathematical process by a finite approximation. Examples include terminating an infinite series and replacing a derivative by a finite-difference formula.

Example 1. (Round-off error) Let

$$f(x) = \frac{x^2 - \frac{1}{9}}{x - \frac{1}{3}}.$$

Using four-digit arithmetic, evaluate $f(0.3334)$ directly and then by first algebraically simplifying the formula.

Solution.
Direct substitution in four-digit arithmetic gives

$$x^2 \approx 0.1112, \qquad \frac{1}{9} \approx 0.1111, \qquad x - \frac{1}{3} \approx 0.3334 - 0.3333 = 0.0001.$$

Hence

$$f(0.3334) \approx \frac{0.1112 - 0.1111}{0.3334 - 0.3333} = \frac{0.0001}{0.0001} = 1.$$

Algebraically however,

$$f(x) = \frac{x^2 - \frac{1}{9}}{x - \frac{1}{3}} = \frac{\left(x - \frac{1}{3}\right)\left(x + \frac{1}{3}\right)}{x - \frac{1}{3}} = x + \frac{1}{3}, \quad \text{for } x \neq \frac{1}{3}$$

Therefore

$$f(0.3334) = 0.3334 + 0.3333 = 0.6667.$$

and the round off error is

$$1 - 0.6667 = 0.3333.$$

The direct formula subtracts nearly equal numbers in both numerator and denominator. This is an example of *catastrophic cancellation*: small rounding errors are magnified by subtraction. □

Example 2. (Truncation error) Recall the Taylor series for e^x:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots.$$

If $1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \cdots + \frac{1}{10!}$ is used to approximate e, identify the truncation error.

Solution.
Since $e = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \cdots$, the omitted tail is

$$R = \frac{1}{11!} + \frac{1}{12!} + \frac{1}{13!} + \cdots.$$

This infinite tail is the truncation error of the finite approximation. □

1.3 Taylor Series and Approximation

Theorem 1. (Taylor's Theorem) *Suppose that* $f \in C^n[a,b]$ *and that* $f^{(n+1)}$ *exists on* $[a,b]$. *Let* $x_0 \in [a,b]$. *For every* $x \in [a,b]$, *there exists a number* $\xi \in (x_0, x)$ *such that*

$$f(x) = P_n(x) + R_n(x), \quad P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x-x_0)^k, \quad R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x-x_0)^{n+1}.$$

where $P_n(x)$ *is the nth-order Taylor polynomial, and* $R_n(x)$ *is the Lagrange remainder.*

The Taylor polynomial $P_n(x)$ is used as an approximation to $f(x)$, while $R_n(x)$ is the associated truncation error. In applications, we want to find the upper bound for the magnitude of the truncation error (as we have learned in Calculus II):

$$|R_n(x)| \le \frac{M}{(n+1)!}|x-x_0|^{n+1},$$

where M can be considered as the absolute maximum of $f^{(n+1)}(x)$ on (x_0, x).

$$\text{i.e.} \quad |f^{(n+1)}(t)| \le M \quad \text{for every } t \in (x_0, x)$$

Example 3. Let $f(x) = \ln(1+x)$. Find the third-order Taylor polynomial $P_3(x)$ about $x_0 = 0$ and use it to estimate $\ln(1.2)$.

Solution.

$$f(x) = \ln(1+x), \qquad f'(x) = \frac{1}{1+x}, \qquad f''(x) = -\frac{1}{(1+x)^2}, \qquad f'''(x) = \frac{2}{(1+x)^3}.$$

At $x_0 = 0$,

$$f(0) = 0, \qquad f'(0) = 1, \qquad f''(0) = -1, \qquad f'''(0) = 2.$$

Thus

$$P_3(x) = \sum_{k=0}^{3} \frac{f^{(k)}(x_0)}{k!}(x-x_0)^k = 0 + \frac{1}{1!}x - \frac{1}{2!}x^2 + \frac{2}{3!}x^3 = x - \frac{x^2}{2} + \frac{x^3}{3}.$$

Then, $\ln(1.2) = \ln(1+0.2) \approx P_3(0.2) = 0.2 - \frac{0.2^2}{2} + \frac{0.2^3}{3} = 0.182666\ldots \approx 0.1827.$ □

Example 4. Estimate an upper bound for the truncation error when $P_3(0.2)$ is used to approximate $\ln(1.2)$.

Solution.
For $f(x) = \ln(1+x)$,

$$f^{(4)}(x) = \frac{d}{dx}f'''(x) = \frac{d}{dx}\frac{2}{(1+x)^3} = -\frac{6}{(1+x)^4}.$$

On the interval $0 \le x \le 0.2$,

$$0 \le x \le 0.2 \implies 1 \le (1+x)^4 \le 1.2^4 \implies \frac{1}{1.2^4} \le \frac{1}{(1+x)^4} \le 1.$$

$$\therefore |f^{(4)}(x)| = \frac{6}{(1+x)^4} \le 6.$$

$$\implies |R_3(0.2)| = \left| \frac{f^{(4)}(x)}{4!}(x-0)^4 \right| \le \frac{6}{4!}|0.2|^4 = \frac{6}{24}(0.0016) = 0.0004.$$

Hence, the upper bound for the truncation error in this case is 0.0004. □

1.4 Number Systems

Humans normally use the decimal (or more technically, the base-10) number system. Computers store information using bits, or binary digits, so machine arithmetic is based on the binary, or base-2, system. Binary digits are often grouped in blocks of four and written in hexadecimal, or base-16, notation.

A very formal definition for a number in a base-b number system is as follows:
For a positional number system with base b,

$$(a_n a_{n-1} \cdots a_0 . a_{-1} a_{-2} \cdots)_b = \sum_{k=-\infty}^{n} a_k b^k,$$

where each digit satisfies $0 \leq a_k < b$.

The following is a conversion table for decimal, binary, and hexadecimal numbers:

Decimal	0	1	2	3	4	5	6	7
Binary	0000	0001	0010	0011	0100	0101	0110	0111
Hexadecimal	0	1	2	3	4	5	6	7
Decimal	8	9	10	11	12	13	14	15
Binary	1000	1001	1010	1011	1100	1101	1110	1111
Hexadecimal	8	9	A	B	C	D	E	F

To convert any decimal integer to a number of any other bases, we perform repeated division and keep track of the remainders as we do so. Then, the converted number is read from bottom to top (Example 5(a)).

Converting to base-10 can be done by summing up the digits multiplied by the base raised to the power of its position number (Example 5(b),(d)).

There also is a trick to convert a base-2 number to other bases of powers of 2. You perhaps have seen this trick in secondary school (if you took ASK) when converting binary, octal (base-8), and hexadecimal numbers. To convert from binary (base-2) to base-2^k, we group k digits together and perform the corresponding conversions to each of the groups of digits (Example 5(c)). In this course, we will only handle decimal, binary, and hexadecimal numbers however.

Example 5. Perform the following conversions.

(a) Convert 1234_{10} to binary.

(b) Convert 10011010010_2 to decimal.

(c) Convert 10011010010_2 to hexadecimal.

(d) Convert $4D2_{16}$ to decimal.

Solution.

(a)

$\div$	quotient	remainder
2	1234	0
2	617	1
2	308	0
2	154	0
2	77	1
2	38	0
2	19	1
2	9	1
2	4	0
2	2	0
2	1	1
2	0	

$\therefore 1234_{10} = 10011010010_2$.

(b) $10011010010_2 = 2^{10} + 2^7 + 2^6 + 2^4 + 2^1 = 1234_{10}$.

(c) Grouping the bits from the right in blocks of four,

$$10011010010_2 = 0100\ 1101\ 0010_2 = 4D2_{16}.$$

(d) $4D2_{16} = 4(16^2) + 13(16) + 2 = 1024 + 208 + 2 = 1234_{10}$.

□

For numbers in bases other than 10 that have a point (e.g. 1.01_2, $A.42_{16}$), we call them fractional base-k numbers.

To convert them to base-10, we separate the digits into 2 groups, the digits before the point and digits after the point. For digits before the point, we repeatedly divide until we get 0 as we have discussed previously. For digits after the point, we repeatedly *multiply* the fractional (or decimal) part while recording the whole number part. Similarly, we stop when the fractional part reaches 0.

As for converting fractional numbers *to* base-10, we sum up the powers of the base according to its position number like before; just that now numbers after the point have a *negative* power.

The trick for base-2^k fractional numbers still work too, we just need to group numbers around the point.

Example 6. Convert 11011.0110_2 to decimal and hexadecimal.

Solution.
For the decimal conversion,

$$\begin{aligned} 11011.0110_2 &= 2^4 + 2^3 + 2^1 + 2^0 + 2^{-2} + 2^{-3} \\ &= 16 + 8 + 2 + 1 + \frac{1}{4} + \frac{1}{8} \\ &= 27.375_{10}. \end{aligned}$$

For the hexadecimal conversion, group into blocks of four around the binary point:

$$11011.0110_2 = 0001\ 1011.0110_2 = 1B.6_{16}.$$

□

We should also consider infinite decimals (repeating and non-repeating) when discussing the conversion algorithm between different number bases. Thus, we will use the idea of significant figures and rounding as we did in base-10. In base-k, we round up if the following digit is $\geq k/2$ and vice versa.

Note: In this course, we start counting significant digits immediately after the point.

Example 7. Convert $\frac{1}{3}$ to a binary number of three significant figures. Find the absolute round-off error.

Solution.

$\times$	fractional part	carry
	1/3	
2	2/3	0
2	1/3	1
2	2/3	0
2	1/3	1
$\vdots$	$\vdots$	$\vdots$

$\therefore \frac{1}{3} = 0.0101\ldots_2$.

Since we want three significant figures,

$$\frac{1}{3} \approx 0.011_2 = \frac{3}{8} = 0.375_{10}.$$

Thus the absolute round-off error is

$$\left|\frac{1}{3} - \frac{3}{8}\right| = \frac{1}{24} = 0.041666\ldots \approx 0.0417.$$

□

We also consider an alternative method in solving the above example.

Solution.

We begin by multiplying $\frac{1}{3}$ by 2^3 since we want 3 significant figures. Thus, we have:

$$\begin{aligned} \frac{1}{3} \times 2^3 &= \frac{8}{3} = 2.67 \approx 3_{10} = 11_2 \\ \implies \frac{1}{3} &= 11_2 \times 2^{-3} = 0.011_2 \end{aligned}$$

And the absolute round-off error is $\left|\frac{1}{3} - (3 \times 2^{-3})\right| \approx 0.0417.$ □

Notice that in the alternative method discussed above, we are able to work with a round number instead of needing to repeatedly multiply fractional parts.

1.5 IEEE Floating-Point Numbers

A computer has only finite memory, so it stores a real number using a finite floating-point representation. Common formats are:

- 32 bits: *single precision*;
- 64 bits: *double precision*.

A commonly used floating-point standard is the **IEEE 754** standard:

$$r = (-1)^s \times 2^q \times m,$$

where s is the sign bit, q is the true exponent, and m is the mantissa.
The true exponent is $q = e - b$, where e is the exponent and b is the bias.

1.5.1 IEEE 754 Single Precision

A normalized single-precision number uses 32-bit memory; one sign bit, eight exponent bits, and twenty-three fraction bits:

s	exponent e (8 bits)	fraction f (23 bits)

For a normalized value,

$$fl(r) = (-1)^s 2^{e-127}(1.f)_2.$$

The bias, b is 127.

Example 8. Write $x = -1234$ in IEEE 754 single-precision floating-point representation and hexadecimal form.

Solution.
Converting 1234_{10} to binary:

$$1234_{10} = 10011010010_2 = 1.0011010010_2 \times 2^{10}.$$

Therefore

$$s = 1, \qquad e = 10 + 127 = 137 = 10001001_2.$$

The fraction field is the part after the leading 1:

$$f = 00110100100000000000000.$$

Hence the 32 bits are

$$\underbrace{1}_{s}\ \underbrace{10001001}_{e}\ \underbrace{00110100100000000000000}_{f}, \text{ or } 11000100100110100100000000000000_2.$$

Grouping into blocks of four gives

$$1100\ 0100\ 1001\ 1010\ 0100\ 0000\ 0000\ 0000 = \boxed{\texttt{C49A4000}}.$$

□

Example 9. Given $x = 4.375_{10} = 100.011_2$, convert x to IEEE 754 single precision and give the answer in hexadecimal form.

Solution.
In scientific notation, we have:

$$100.011_2 = 1.00011_2 \times 2^2.$$

Thus

$$s = 0, \quad e = 2 + 127 = 129 = 10000001_2, \quad f = 00011000000000000000000.$$

The bit pattern is

$$0\ 10000001\ 00011000000000000000000.$$

Grouping into hexadecimal digits,

$$0100\ 0000\ 1000\ 1100\ 0000\ 0000\ 0000\ 0000 = \boxed{\texttt{408C0000}}.$$

□

1.5.2 IEEE 754 Double Precision

A normalized double-precision number uses one sign bit, eleven exponent bits, and fifty-two fraction bits:

s	exponent e (11 bits)	fraction f (52 bits)

For a normalized value,

$$fl(r) = (-1)^s 2^{e-1023} (1.f)_2.$$

The bias, b is 1023.

Example 10. Convert the following IEEE 754 double-precision machine number to decimal:

$$0100000000111011100100010000000000000000000000000000000000000000$$

Solution.
Separate the fields:

$$s = 0, \qquad e = 10000000011_2 = 1027,$$

and the fraction begins

$$f = 1011100100010000\ldots.$$

The true exponent is

$$q = e - 1023 = 1027 - 1023 = 4.$$

The significand is

$$\begin{aligned}(1.f)_2 &= 1.101110010001_2 \\ &= 1 + 2^{-1} + 2^{-3} + 2^{-4} + 2^{-5} + 2^{-8} + 2^{-12} \\ &= 1.722900390625.\end{aligned}$$

Therefore
$$r = 2^4(1.722900390625) = \boxed{27.56640625}.$$

$\square$

Remark 1. IEEE 754 also reserves exponent patterns for zero, subnormal numbers, infinities and NaN (Not a Number). The formulas above apply to normalized finite numbers.

1.6 Algorithms, Convergence and Accuracy

An algorithm is a procedure that describes a finite sequence of steps performed in a specified order to solve a problem or produce a good approximation of the solution of said problem.

Many numerical algorithms are iterative. Starting from an initial approximation x_0, they generate a sequence
$$\{x_n\} = \{x_0, x_1, x_2, \ldots\}$$
by repeatedly applying an update formula $f(x)$ s.t. $x_{i+1} = f(x_i)$. Main concepts are:

- **initial condition**: the starting approximation or initial guess x_0;
- **iteration**: applying the algorithm $x_{i+1} = f(x_i)$;
- **convergence**: for a sufficiently large N, x_N approach a limiting value;
- **stopping criterion**: a rule for deciding when the approximation is sufficiently accurate.

Common stopping criteria include
$$|x_n - x_{n-1}| < \epsilon, \qquad \left|\frac{x_n - x_{n-1}}{x_n}\right| < \epsilon, \qquad |f(x_n)| < \epsilon,$$
where ϵ denotes a sufficiently small error. Sometimes, we also include a maximum allowed number of iterations.

Example 11. Find an approximate root of
$$x^2 - 5x + 6 = 0.$$

Solution.
An algorithm to estimate the root of $x^2 - 5x + 6 = 0$ is the Newton-Raphson Method (to be covered in Chapter 3).
We first choose an initial guess x_0. Let $x_0 = 0$.
Then let
$$f(x) = x^2 - 5x + 6, \qquad f'(x) = 2x - 5.$$
Then, we improve the guess by Newton's formula:
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = x_n - \frac{x_n^2 - 5x_n + 6}{2x_n - 5}.$$

Repeatedly applying the formula until desirable accuracy, we have:

n	x_n
0	1.000000
1	1.666667
2	1.933333
3	1.996078
4	1.999985
5	2.000000

Thus, the sequence converges to the root $x = 2$.

Checking the exact solution, our above equation factors as:

$$x^2 - 5x + 6 = (x - 2)(x - 3),$$

so the exact roots are 2 and 3.

Looking at our approximation and accuracy at $n = 4$, the absolute error relative to the exact root is approximately

$$|2 - x_4| = |2 - 1.999985| \approx 1.5 \times 10^{-5}.$$

A practical implementation would stop once a prescribed stopping criteria (e.g. a tolerance or error, ϵ, or usually an approximation of such an error) is met, or once a given maximum iteration count is reached. $\square$

Remark 2. Convergence is not guaranteed merely because an iterative formula can be written down. The choice of algorithm, initial approximation and stopping criterion can all affect whether the computed sequence converges and how quickly it does so.

In the upcoming chapters, we will explore a handful of algorithms to approximate a solution to a few different types of problems.

Tutorial 1

1. Let $f(x) = \sqrt{x+1}$.
   (a) Find the third-order Taylor polynomial $P_3(x)$ about $x_0 = 0$.
   (b) Use $P_3(x)$ to approximate $\sqrt{0.5}$ and $\sqrt{1.25}$.
   (c) Find the absolute errors of the approximations in part (b).
2. Let $f(x) = \cos x$.
   (a) Find the third-order Taylor polynomial $P_3(x)$ for the function $f(x)$ about $x_0 = 0$.
   (b) Find an upper bound for the truncation error when approximating $\cos(0.01)$ using the polynomial obtained in part (a).
3. (a) Convert the decimal number 5432_{10} to a binary number.
   (b) Convert the binary number 1010100111000_2 to a decimal number.
   (c) Convert the binary number 1010100111000_2 to a hexadecimal number.

(d) Convert the hexadecimal number 1538_{16} to a decimal number.

4. Convert each of the following binary numbers into decimal and hexadecimal numbers.
   (a) 110011.011010_2
   (b) 11100010.100111_2

5. Convert $\dfrac{1}{3}$ to a binary number of eight significant digits. Find the round-off error.

6. Convert the decimal number 12345_{10} to the IEEE 754 single-precision floating-point format.

7. Given that

$$x = 10001110011.00011_2,$$

   convert x to the IEEE 754 single-precision floating-point representation. Express the answer in hexadecimal form.

8. Convert the machine number

$$01000001101110010000000000000000$$

   to a decimal number.

Chapter 2

NUMERICAL LINEAR ALGEBRA

Note: If you are already familiar with the concepts taught in Linear Algebra (or perhaps Basic Mathematics if you are an FM student), you can directly skip to the summary before Section 2.2.
However, throughout this section, I assume you have at least seen some of the basic notations discussed in the Linear Algebra (or Basic Mathematics).

2.1 Linear Systems and Direct Methods

A system of m linear equations in n unknowns can be written as

$$A\boldsymbol{x} = \boldsymbol{b},$$

where $A = [a_{ij}] \in \mathbb{R}^{m\times n}$ is the coefficient matrix, $\boldsymbol{x} = [x_1, \ldots, x_n]^T$ is the vector of unknowns, and $\boldsymbol{b} = [b_1, \ldots, b_m]^T$ is the right-hand side. In this course we are mainly interested in square systems, $m = n$.

We have previously learned a few direct methods to simplify linear systems so that an exact solution can easily be found. Gaussian elimination and matrix factorisation are such methods.

2.1.1 Gaussian Elimination

Gaussian elimination transforms the augmented matrix $[A \mid \boldsymbol{b}]$ to row-echelon form (i.e. into an upper-triangular system), or in matrix notation $[U \mid \boldsymbol{b}^*]$. For a pivot in row k, the entries below the pivot are removed using elementary row operations as follows:

$$R_j - m_{jk}R_k \to R_j, \qquad m_{jk} = \frac{a_{jk}^{(k)}}{a_{kk}^{(k)}}, \qquad j = k+1, \ldots, n.$$

For reference, $a_{ij}^{(k)}$ refers to the element in the i-th row and j-th column while eliminating entries below the pivot in the k-th row. Once an upper-triangular system $U\boldsymbol{x} = \boldsymbol{b}^*$ is obtained, backward substitution gives

$$x_i = \frac{1}{u_{ii}}\left(c_i - \sum_{j=i+1}^{n} u_{ij}x_j\right), \qquad i = n, n-1, \ldots, 1.$$

Algorithm. (Gaussian elimination)

(i) Form the augmented matrix $[A \mid \boldsymbol{b}]$.

(ii) For $k = 1, \ldots, n-1$, use row operations to eliminate every entry below the pivot $a_{kk}^{(k)}$.

(iii) Solve the resulting upper-triangular system by backward substitution.

Example 12. Solve the following system of equations with Gaussian elimination:

$$\begin{array}{rcrcrcrcr} x_1 & + & x_2 & + & & & 3x_4 & = & 4, \\ 2x_1 & + & x_2 & - & x_3 & + & x_4 & = & 1, \\ 3x_1 & - & x_2 & - & x_3 & + & 2x_4 & = & -3, \\ -x_1 & + & 2x_2 & + & 3x_3 & - & x_4 & = & 4. \end{array}$$

Solution.
Form the augmented matrix:

$$\left[\begin{array}{rrrr|r} 1 & 1 & 0 & 3 & 4 \\ 2 & 1 & -1 & 1 & 1 \\ 3 & -1 & -1 & 2 & -3 \\ -1 & 2 & 3 & -1 & 4 \end{array}\right].$$

Then,

$$\left[\begin{array}{rrrr|r} 1 & 1 & 0 & 3 & 4 \\ 2 & 1 & -1 & 1 & 1 \\ 3 & -1 & -1 & 2 & -3 \\ -1 & 2 & 3 & -1 & 4 \end{array}\right] \xrightarrow[R_4 + R_1 \to R_4]{\substack{R_2 - 2R_1 \to R_2 \\ R_3 - 3R_1 \to R_3}} \left[\begin{array}{rrrr|r} 1 & 1 & 0 & 3 & 4 \\ 0 & -1 & -1 & -5 & -7 \\ 0 & -4 & -1 & -7 & -15 \\ 0 & 3 & 3 & 2 & 8 \end{array}\right]$$

$$\xrightarrow[R_4 + 3R_2 \to R_4]{R_3 - 4R_2 \to R_3} \left[\begin{array}{rrrr|r} 1 & 1 & 0 & 3 & 4 \\ 0 & -1 & -1 & -5 & -7 \\ 0 & 0 & 3 & 13 & 13 \\ 0 & 0 & 0 & -13 & -13 \end{array}\right].$$

Backward substitution yields

$$x_4 = \frac{-13}{-13} = 1, \quad x_3 = \frac{13 - 13(1)}{3} = 0,$$

$$x_2 = \frac{-7 + 5(1) + 1(0)}{-1} = 2, \quad x_1 = \frac{4 - 3(1) - 0(0) - 2}{1} = -1.$$

Thus

$$\boldsymbol{x} = [-1, 2, 0, 1]^T.$$

□

In most cases, Gaussian elimination is the preferred way to solve a linear system $A\boldsymbol{x} = \boldsymbol{b}$. But there are two problems with Gaussian Elimination, which we will discuss in the following subsections.

2.1.2 Pivoting Strategies

We have previously abstracted the steps for Gaussian Elimination. However, if we were to follow the steps blindly, we may run into a few problems (division by zero or numerical instabilities).

A zero pivot prevents the next elimination step (since it would cause division by zero if we were to eliminate blindly). Even a nonzero but very small pivot can magnify round-off error because the multipliers m_{jk} become large.

Example 13. Zero pivot Solve

$$\begin{cases} 0x_1 + x_2 = 1 \\ x_1 + x_2 = 2 \end{cases}$$

Solution.
After interchanging the two equations, the system becomes

$$x_1 + x_2 = 2, \qquad x_2 = 1,$$

and thus $x_2 = 1$ and $x_1 = 1$. $\square$

Example 14. Small pivot Consider

$$\begin{cases} \epsilon x_1 + x_2 = 1 \\ x_1 + x_2 = 2 \end{cases}$$

where ϵ is very small. We will see why elimination without pivoting may be unreliable.

Solution.
Using the first equation as pivot gives the multiplier $1/\epsilon$, which is very large. The second equation becomes

$$\left(1 - \frac{1}{\epsilon}\right) x_2 = 2 - \frac{1}{\epsilon} \implies x_2 = \frac{2 - \epsilon^{-1}}{1 - \epsilon^{-1}} \approx 1.$$

Substituting back into the first equation, we will get:

$$x_1 = \epsilon^{-1}(1 - x_2) \approx \epsilon^{-1}(1 - 1) = 0.$$

It is clear that this solution became unreliable when we executed Gaussian Elimination numerically.
(Substituting our solution $x_1 = 0, x_2 = 1$ into Equation 2 yields a contradiction.) $\square$

Definition. (Partial pivoting) At a given step k, partial pivoting exchanges row k (the original pivot row) with a row below it with the largest leading element. Mathematically, we want a row $p \geq k$ for which

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|.$$

Partial pivoting is applied in every step of elimination so that we can get a safe and accurate solution.

Example 15. Apply Gaussian elimination with partial pivoting to solve

$$\begin{cases} \epsilon x_1 + x_2 = 1 \\ x_1 + x_2 = 2 \end{cases}$$

when ϵ is small.

Solution.
Since $|1| > |\epsilon|$, exchange the two equations:

$$\begin{cases} x_1 + x_2 = 2 \\ \epsilon x_1 + x_2 = 1 \end{cases}$$

Then applying row operation $R_2 - \epsilon R_1 \to R_2$ gives

$$(1-\varepsilon)x_2 = 1 - 2\varepsilon,$$

and hence

$$x_2 = \frac{1-2\varepsilon}{1-\varepsilon} \approx 1, \qquad x_1 = 2 - x_2 \approx 1.$$

$\square$

2.1.3 Matrix Inversion

If A is nonsingular, its inverse can be computed by applying Gauss-Jordan elimination (i.e. reducing a matrix to r.r.e.f., reduced row-echelon form) to the block matrix $[A \mid I]$ until the left block becomes I:

$$[A \mid I] \longrightarrow [I \mid A^{-1}].$$

However, when considering a square matrix of order n, finding the inverse using this method would mean applying Gauss-Jordan Elimination to an $n \times 2n$ augmented matrix.

The issue with speed could be resolved for certain classes of problems. The table below shows the operation counts for Gaussian Elimination:

	Multiplication/Divisions	Additions/Subtractions
Elimination Steps	$\frac{2n^3+3n^2-5n}{6}$	$\frac{n^3-n}{3}$
Backward Substitution	$\frac{n^2+n}{2}$	$\frac{n^2-n}{2}$

From here, we see that the number of operations to do backward substitution is significantly smaller than doing the elimination steps. Just considering a linear system with a 1000×1000 matrix, we would need 667,165,500 operations for elimination, but only 1,000,000 operations (0.15%) to perform backward substitution.

Hence, for efficiency, when we compute the inverse for a matrix A, we will reduce $[A|I]$ to $[U|Y]$ where U is an upper triangular matrix (or as we know the r.e.f. of A) and Y is I after performing elimination. Then, we will individually obtain the columns of A^{-1} through solving $U\boldsymbol{x}_i = \boldsymbol{y}_i$ where $\boldsymbol{y}_i$ denotes the i-th column of Y.

Example 16. Use Gaussian Elimination followed by backward substitution to find the inverse of

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 1 & 0 \\ -1 & 1 & 2 \end{bmatrix}.$$

Solution.

$$[A|I] = \left[\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 0 & 1 & 0 \\ -1 & 1 & 2 & 0 & 0 & 1 \end{array}\right] \xrightarrow[R_3 + R_1 \to R_3]{R_2 - 2R_1 \to R_2} \left[\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & -3 & 2 & -2 & 1 & 0 \\ 0 & 3 & 1 & 1 & 0 & 1 \end{array}\right]$$

$$\xrightarrow{R_3 + R_2 \to R_3} \left[\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & -3 & 2 & -2 & 1 & 0 \\ 0 & 0 & 3 & -1 & 1 & 1 \end{array}\right] = [U|Y]$$

$$\begin{bmatrix} 1 & 2 & -1 \\ 0 & -3 & 2 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ -1 \end{bmatrix}$$

$$\implies x_3 = -\frac{1}{3},\ x_2 = \frac{-2 - 2(-\frac{1}{3})}{-3} = \frac{4}{9},\ x_1 = \frac{1 + 1(-\frac{1}{3}) - 2(\frac{4}{9})}{1} = -\frac{2}{9}.$$

$$\begin{bmatrix} 1 & 2 & -1 \\ 0 & -3 & 2 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\implies x_3 = \frac{1}{3},\ x_2 = \frac{1 - 2(\frac{1}{3})}{-3} = -\frac{1}{9},\ x_1 = \frac{0 + 1(\frac{1}{3}) - 2(-\frac{1}{9})}{1} = \frac{5}{9}.$$

$$\begin{bmatrix} 1 & 2 & -1 \\ 0 & -3 & 2 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\implies x_3 = \frac{1}{3},\ x_2 = \frac{0 - 2(\frac{1}{3})}{-3} = \frac{2}{9},\ x_1 = \frac{0 + 1(\frac{1}{3}) - 2(\frac{2}{9})}{1} = -\frac{1}{9}.$$

$$\therefore A^{-1} = \begin{bmatrix} -\frac{2}{9} & \frac{5}{9} & -\frac{1}{9} \\ \frac{4}{9} & -\frac{1}{9} & \frac{2}{9} \\ -\frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}.$$

□

2.1.4 Matrix Factorisation

Gaussian elimination is the principal tool in the direct solution of linear system of equations. Previously, we have learnt that the steps used for elimination can also be used to factor a matrix. Suppose $A = LU$, where L is lower triangular and U is upper triangular. Then

$$A\boldsymbol{x} = \boldsymbol{b} \quad \iff \quad L(U\boldsymbol{x}) = \boldsymbol{b}.$$

Let $U\boldsymbol{x} = \boldsymbol{y}$. The system is solved in two inexpensive triangular stages:

$$L\boldsymbol{y} = \boldsymbol{b} \quad \text{(forward substitution)}, \qquad U\boldsymbol{x} = \boldsymbol{y} \quad \text{(backward substitution)}.$$

The factorisation costs roughly as much as one Gaussian elimination, but it can be reused for every new right-hand side.

Example 17. Factorise

$$A = \begin{pmatrix} 1 & 1 & 0 & 3 \\ 2 & 1 & -1 & 1 \\ 3 & -1 & -1 & 2 \\ -1 & 2 & 3 & -1 \end{pmatrix}$$

into LU form and solve $A\boldsymbol{x} = \boldsymbol{b}$ for $\boldsymbol{b} = (1, 1, -3, 4)^\mathsf{T}$.

Solution.
The elimination multipliers give

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 3 & 4 & 1 & 0 \\ -1 & -3 & 0 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 1 & 1 & 0 & 3 \\ 0 & -1 & -1 & -5 \\ 0 & 0 & 3 & 13 \\ 0 & 0 & 0 & -13 \end{pmatrix}.$$

First solve $L\boldsymbol{y} = \boldsymbol{b}$:

$$y_1 = 1, \quad y_2 = -1, \quad y_3 = -2, \quad y_4 = 2.$$

Then solve $U\boldsymbol{x} = \boldsymbol{y}$:

$$x_4 = -\frac{2}{13}, \quad x_3 = 0, \quad x_2 = \frac{23}{13}, \quad x_1 = -\frac{4}{13}.$$

Therefore

$$\boldsymbol{x} = \left(-\frac{4}{13}, \frac{23}{13}, 0, -\frac{2}{13}\right)^\mathsf{T}.$$

□

2.2 Iterative Methods for Linear Systems

Iterative methods begin with an initial vector $\boldsymbol{x}^{(0)}$ and generate

$$\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots$$

that may converge to the exact solution. They are especially attractive for large sparse systems.

Write

$$A = D + L + U,$$

where D contains the diagonal entries of A, L the entries below the diagonal, and U the entries above the diagonal.

2.2.1 Jacobi Method

The Jacobi iteration is

$$\boldsymbol{x}^{(k+1)} = D^{-1}\Big(\boldsymbol{b} - (L+U)\boldsymbol{x}^{(k)}\Big).$$

Componentwise,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j\neq i} a_{ij}x_j^{(k)}\right).$$

Every new component uses only values from the previous iteration, so all corrections may be computed simultaneously.

2.2.2 Gauss–Seidel Method

The Gauss–Seidel iteration is

$$(D+L)\boldsymbol{x}^{(k+1)} = \boldsymbol{b} - U\boldsymbol{x}^{(k)}.$$

Componentwise,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right).$$

Newly computed values are used immediately, so Gauss–Seidel often converges faster than Jacobi.

2.2.3 Successive Over-Relaxation

The SOR update is a weighted Gauss–Seidel correction:

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right),$$

where $0 < \omega < 2$. When $\omega = 1$, SOR reduces to Gauss–Seidel. Values $0 < \omega < 1$ under-relax the iteration, while $1 < \omega < 2$ over-relax it.

Definition. (Strict diagonal dominance) A square matrix $A = [a_{ij}]$ is strictly diagonally dominant by rows if

$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|, \qquad i = 1,\dots,n.$$

Example 18. Show that

$$A = \begin{pmatrix} 2 & -7 & 4 \\ 8 & 1 & 6 \\ -3 & 5 & 9 \end{pmatrix}$$

is not strictly diagonally dominant.

Solution.
The first row already fails the test because

$$|2| = 2 < |-7| + |4| = 11.$$

The second row also fails because $|1| < |8| + |6|$. Hence A is not strictly diagonally dominant. $\square$

Theorem 2. (Convergence under diagonal dominance) *If A is strictly diagonally dominant, then the Jacobi and Gauss–Seidel iterations converge to the unique solution of $A\boldsymbol{x} = \boldsymbol{b}$ for every initial vector $\boldsymbol{x}^{(0)}$.*

A common stopping test is

$$\max_i |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon.$$

A residual test such as $\left\|\boldsymbol{b} - A\boldsymbol{x}^{(k)}\right\|_\infty < \varepsilon$ is often more informative because it measures how well the computed vector satisfies the original equations.

Example 19. Use the Jacobi method, starting from $\boldsymbol{x}^{(0)} = \mathbf{0}$, to solve

$$\begin{aligned} 20x_1 + x_2 - x_3 &= 17, \\ x_1 - 10x_2 + x_3 &= 13, \\ -x_1 + x_2 + 10x_3 &= 18. \end{aligned}$$

Stop when every component changes by less than 0.0002.

Solution.
The iteration equations are

$$x_1^{(k+1)} = \frac{17 - x_2^{(k)} + x_3^{(k)}}{20},$$

$$x_2^{(k+1)} = \frac{x_1^{(k)} + x_3^{(k)} - 13}{10}, \qquad x_3^{(k+1)} = \frac{18 + x_1^{(k)} - x_2^{(k)}}{10}.$$

The iterates are

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0	0	0
1	0.85000	−1.30000	1.80000
2	1.00500	−1.03500	2.01500
3	1.00250	−0.99800	2.00400
4	1.00010	−0.99935	2.00005
5	0.99997	−0.99998	1.99994
6	1.00000	−1.00001	2.00000

At iteration 6 all changes are below 0.0002. Thus

$$\boldsymbol{x} \approx (1, -1, 2)^\mathsf{T}.$$

$\square$

Example 20. Solve the same system by the Gauss–Seidel method with the same initial vector and stopping criterion.

Solution.

Using the newest available values gives

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0	0	0
1	0.85000	−1.21500	2.00650
2	1.01108	−0.99824	2.00093
3	0.99996	−0.99991	1.99999
4	0.99999	−1.00000	2.00000

The tolerance is met after four iterations, so $\boldsymbol{x} \approx (1, -1, 2)^{\mathsf{T}}$. □

Example 21. Use two SOR iterations with $\omega = 1.25$ and $\boldsymbol{x}^{(0)} = \mathbf{0}$ for

$$\begin{aligned} 3x_1 - x_2 + x_3 &= -1, \\ -x_1 + 3x_2 - x_3 &= 7, \\ x_1 - x_2 + 3x_3 &= -7. \end{aligned}$$

Solution.
The first SOR sweep gives

$$\boldsymbol{x}^{(1)} = (-0.41667,\ 2.74306,\ -1.60012)^{\mathsf{T}}.$$

Using these values in the second sweep gives

$$\boldsymbol{x}^{(2)} = (1.49715,\ 2.18800,\ -2.22878)^{\mathsf{T}}.$$

Thus after two iterations,

$$x_1 \approx 1.4971, \qquad x_2 \approx 2.1880, \qquad x_3 \approx -2.2288.$$

□

2.2.4 Conjugate Gradient Method

For a symmetric positive-definite matrix A, solving $A\boldsymbol{x} = \boldsymbol{b}$ is equivalent to minimising

$$\phi(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^{\mathsf{T}} A\boldsymbol{x} - \boldsymbol{x}^{\mathsf{T}}\boldsymbol{b}.$$

The conjugate gradient method constructs mutually A-conjugate search directions and, in exact arithmetic, reaches the exact solution in at most n steps.

Algorithm. (Conjugate gradient) Choose $\boldsymbol{x}_0$. Set

$$\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0, \qquad \boldsymbol{p}_0 = \boldsymbol{r}_0.$$

For $k = 0, 1, 2, \ldots$, compute

$$\alpha_k = \frac{\boldsymbol{r}_k^{\mathsf{T}}\boldsymbol{r}_k}{\boldsymbol{p}_k^{\mathsf{T}} A\boldsymbol{p}_k}, \qquad \boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{p}_k,$$

$$\boldsymbol{r}_{k+1} = \boldsymbol{r}_k - \alpha_k A\boldsymbol{p}_k, \qquad \beta_k = \frac{\boldsymbol{r}_{k+1}^{\mathsf{T}}\boldsymbol{r}_{k+1}}{\boldsymbol{r}_k^{\mathsf{T}}\boldsymbol{r}_k},$$
$$\boldsymbol{p}_{k+1} = \boldsymbol{r}_{k+1} + \beta_k\boldsymbol{p}_k.$$

Example 22. Apply conjugate gradient with $\boldsymbol{x}_0 = \boldsymbol{0}$ to

$$A = \begin{pmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 24 \\ 30 \\ -24 \end{pmatrix}.$$

Solution.
The successive approximations are

k	x_1	x_2	x_3
0	0	0	0
1	3.52577	4.40722	−3.52577
2	2.85801	4.14897	−4.95422
3	3.00000	4.00000	−5.00000

The residual after the third step is zero to round-off accuracy, giving

$$\boldsymbol{x} = (3, 4, -5)^{\mathsf{T}}.$$

□

2.3 Eigenvalues and Eigenvectors

Definition. (Eigenvalue and eigenvector) A scalar λ is an eigenvalue of an $n \times n$ matrix A if there exists a nonzero vector $\boldsymbol{v}$ such that

$$A\boldsymbol{v} = \lambda\boldsymbol{v}.$$

The vector $\boldsymbol{v}$ is an eigenvector corresponding to λ.

Eigenvalues satisfy the characteristic equation

$$\det(A - \lambda I) = 0.$$

Once λ is known, its eigenvectors are the nonzero solutions of $(A - \lambda I)\boldsymbol{v} = \boldsymbol{0}$.

Example 23. For

$$A = \begin{pmatrix} 4 & 1 \\ 3 & 2 \end{pmatrix}, \qquad \boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

verify that $\boldsymbol{v}$ is an eigenvector and find its eigenvalue.

Solution.

$$A\boldsymbol{v} = \begin{pmatrix} 5 \\ 5 \end{pmatrix} = 5\begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Hence $\boldsymbol{v}$ is an eigenvector corresponding to $\lambda = 5$. □

2.3.1 Gershgorin Discs

Theorem 3. (Gershgorin's theorem) *Every eigenvalue of $A = [a_{ij}]$ lies in at least one disc*

$$|z - a_{ii}| \le r_i, \qquad r_i = \sum_{j \ne i} |a_{ij}|.$$

For a real symmetric matrix, all eigenvalues are real, so the discs may be viewed as intervals on the real line.

Example 24. Find Gershgorin intervals for

$$A = \begin{pmatrix} 7 & 4 & -4 \\ 4 & -8 & -1 \\ -4 & -1 & -8 \end{pmatrix}.$$

Solution.
The row radii are

$$r_1 = 8, \qquad r_2 = 5, \qquad r_3 = 5.$$

Thus the intervals are

$$[-1, 15], \qquad [-13, -3], \qquad [-13, -3].$$

The first interval is disjoint from the other two, so one eigenvalue lies in $[-1, 15]$ and the remaining two lie in $[-13, -3]$. □

2.3.2 Power Method

Definition. (Dominant eigenvalue) An eigenvalue λ_1 is dominant if

$$|\lambda_1| > |\lambda_j| \qquad \text{for every } j \ne 1.$$

If A is diagonalizable and the initial vector has a nonzero component in the dominant eigendirection, then

$$A^k \boldsymbol{x}_0$$

becomes increasingly parallel to a dominant eigenvector. The corresponding eigenvalue may be estimated by the Rayleigh quotient

$$\rho(\boldsymbol{x}) = \frac{\boldsymbol{x}^\mathsf{T} A \boldsymbol{x}}{\boldsymbol{x}^\mathsf{T} \boldsymbol{x}}.$$

The convergence rate is governed chiefly by $|\lambda_2/\lambda_1|$.

Example 25. Determine whether a dominant eigenvalue exists in each case:

(a) $-5, 4, 2, -3$;

(b) $2, -5, 5$.

Solution.
In (a), -5 is dominant because its modulus is strictly largest. In (b), both -5 and 5 have modulus 5, so no unique dominant eigenvalue exists. $\square$

Example 26. Let

$$A = \begin{pmatrix} 4 & -2 \\ 3 & -1 \end{pmatrix}, \qquad \boldsymbol{x}_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Perform six power iterations and estimate the dominant eigenpair.

Solution.
Successive products are

$$\begin{aligned} \boldsymbol{x}_1 &= (4,3)^\mathsf{T}, \\ \boldsymbol{x}_2 &= (10,9)^\mathsf{T}, \\ \boldsymbol{x}_3 &= (22,21)^\mathsf{T}, \\ \boldsymbol{x}_4 &= (46,45)^\mathsf{T}, \\ \boldsymbol{x}_5 &= (94,93)^\mathsf{T}, \\ \boldsymbol{x}_6 &= (190,189)^\mathsf{T}. \end{aligned}$$

The direction approaches $(1,1)^\mathsf{T}$. The Rayleigh quotient at the sixth iterate is

$$\rho(\boldsymbol{x}_6) = \frac{\boldsymbol{x}_6^\mathsf{T} A \boldsymbol{x}_6}{\boldsymbol{x}_6^\mathsf{T} \boldsymbol{x}_6} \approx 2.0132.$$

Thus the dominant eigenvalue is approximately 2.0132 and a corresponding eigenvector is approximately $(190,189)^\mathsf{T}$. The exact dominant eigenpair is $\lambda = 2$ with eigenvector $(1,1)^\mathsf{T}$. $\square$

To prevent entries from growing too large, scale after every multiplication:

$$\boldsymbol{y}_k = A\boldsymbol{x}_{k-1}, \qquad \boldsymbol{x}_k = \frac{\boldsymbol{y}_k}{\max_i |y_{k,i}|}.$$

Example 27. Repeat the previous example using power iteration with scaling.

Solution.
After six scaled iterations,

$$\boldsymbol{x}_6 \approx (1, 0.9947)^\mathsf{T}.$$

The Rayleigh quotient gives

$$\lambda_{\max} \approx 2.0133.$$

Scaling changes only the size of the iterates, not their direction. $\square$

2.3.3 Inverse and Shifted Inverse Power Methods

If A is nonsingular, the eigenvalues of A^{-1} are $1/\lambda_i$. Hence power iteration applied to A^{-1} finds the eigenvalue of A having smallest modulus. In practice one solves

$$A\boldsymbol{y}_k = \boldsymbol{x}_{k-1}$$

rather than explicitly forming A^{-1}.

Example 28. Let

$$A = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}, \qquad \boldsymbol{x}_0 = (-0.7, 1)^{\mathsf{T}}.$$

Use two inverse power iterations with scaling to approximate the smaller eigenvalue.

Solution.
Here

$$A^{-1} = \begin{pmatrix} -0.1 & 0.3 \\ 0.4 & -0.2 \end{pmatrix}.$$

The first scaled iterate is approximately

$$\boldsymbol{x}_1 = (0.7708, -1)^{\mathsf{T}}, \qquad \lambda^{(1)} \approx -1.9952.$$

The second is

$$\boldsymbol{x}_2 = (-0.7419, 1)^{\mathsf{T}}, \qquad \lambda^{(2)} \approx -2.0020.$$

The estimated percentage change is about 0.34%. Thus the smaller eigenvalue is approximately -2.0020. □

To find an eigenvalue near a prescribed shift a, apply inverse iteration to $A - aI$. If β is the dominant eigenvalue of $(A - aI)^{-1}$, then

$$\lambda \approx a + \frac{1}{\beta}.$$

Example 29. Use two shifted inverse power iterations with $a = 6$ and $\boldsymbol{x}_0 = (1, 0)^{\mathsf{T}}$ to find the eigenvalue of

$$A = \begin{pmatrix} 7 & -2 \\ 3 & -1 \end{pmatrix}$$

nearest to 6.

Solution.
Since

$$(A - 6I)^{-1} = \begin{pmatrix} 7 & -2 \\ 3 & -1 \end{pmatrix},$$

the scaled iterates are approximately

$$\boldsymbol{x}_1 = (1, 0.4286)^{\mathsf{T}}, \qquad \boldsymbol{x}_2 = (1, 0.4186)^{\mathsf{T}}.$$

At the second step the Rayleigh estimate for $(A - 6I)^{-1}$ is $\beta \approx 6.16337$. Hence

$$\lambda \approx 6 + \frac{1}{6.16337} = 6.1622.$$

□

2.4 Householder Reduction and the QR Algorithm

Definition. (Householder transformation) Let $\boldsymbol{w} \in \mathbb{R}^n$ satisfy $\boldsymbol{w}^\mathsf{T}\boldsymbol{w} = 1$. The matrix

$$P = I - 2\boldsymbol{w}\boldsymbol{w}^\mathsf{T}$$

is a Householder transformation.

A Householder matrix is symmetric and orthogonal:

$$P^\mathsf{T} = P, \qquad P^{-1} = P.$$

It reflects vectors across the hyperplane orthogonal to $\boldsymbol{w}$. Carefully chosen Householder transformations can introduce zeros below the first subdiagonal of a symmetric matrix while preserving symmetry and eigenvalues. Repeating the process reduces a symmetric matrix to tridiagonal form.

Example 30. Reduce

$$A = \begin{pmatrix} 4 & 1 & -2 & 2 \\ 1 & 2 & 0 & 1 \\ -2 & 0 & 3 & -2 \\ 2 & 1 & -2 & -1 \end{pmatrix}$$

to tridiagonal form using Householder transformations.

Solution.
A sequence of orthogonal similarity transformations produces

$$T = \begin{pmatrix} 4 & -3 & 0 & 0 \\ -3 & \frac{10}{3} & -\frac{5}{3} & 0 \\ 0 & -\frac{5}{3} & -\frac{33}{25} & \frac{68}{75} \\ 0 & 0 & \frac{68}{75} & \frac{149}{75} \end{pmatrix}.$$

Because every step has the form PAP^T, the tridiagonal matrix T is similar to A and has the same eigenvalues. ☐

For a symmetric tridiagonal matrix, the QR algorithm repeatedly performs

$$A^{(k)} = Q^{(k)}R^{(k)}, \qquad A^{(k+1)} = R^{(k)}Q^{(k)}.$$

Since

$$A^{(k+1)} = (Q^{(k)})^\mathsf{T} A^{(k)} Q^{(k)},$$

all iterates are similar and have the same eigenvalues. Under suitable conditions the off-diagonal entries tend to zero, and the diagonal entries approach the eigenvalues.

Example 31. Perform one QR iteration for

$$A = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{pmatrix}.$$

Solution.

One QR factorisation is $A = QR$, with

$$Q \approx \begin{pmatrix} 0.94868 & -0.29409 & 0.11625 \\ 0.31623 & 0.88226 & -0.34874 \\ 0 & 0.36761 & 0.92998 \end{pmatrix},$$

$$R \approx \begin{pmatrix} 3.16228 & 1.89737 & 0.31623 \\ 0 & 2.72029 & 1.98560 \\ 0 & 0 & 2.43270 \end{pmatrix}.$$

Therefore

$$A^{(2)} = RQ \approx \begin{pmatrix} 3.60000 & 0.86023 & 0 \\ 0.86023 & 3.12973 & 0.89753 \\ 0 & 0.89753 & 2.27027 \end{pmatrix}.$$

This matrix is symmetric and similar to A, but its off-diagonal entries have begun to change toward a more diagonal form. $\square$

Tutorial 2

Chapter 3

SOLUTION OF EQUATIONS

3.1 Roots and Locating Roots

Definition. (Root or zero) A number r is a root of the equation $f(x) = 0$ if

$$f(r) = 0.$$

The terms *root*, *solution* and *zero of f* are used interchangeably.

Linear and quadratic equations have explicit formulas. For example,

$$ax + b = 0 \quad \Longrightarrow \quad x = -\frac{b}{a}, \qquad a \neq 0,$$

and

$$ax^2 + bx + c = 0 \quad \Longrightarrow \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

For general nonlinear equations, exact formulas are usually unavailable and numerical iteration is required.

Example 32. Find all roots of $x^3 - 4x = 0$.

Solution.
Factor:

$$x^3 - 4x = x(x^2 - 4) = x(x - 2)(x + 2).$$

Hence the roots are -2, 0 and 2. $\square$

Theorem 4. (Intermediate value theorem) *Let f be continuous on $[a, b]$. If $f(a)$ and $f(b)$ have opposite signs, then there exists at least one $r \in (a, b)$ such that $f(r) = 0$.*

The theorem guarantees existence but not uniqueness. It is the theoretical basis of bracketing methods.

Example 33. Show that $x = \cos x$ has at least one solution in $(0, \pi/2)$.

Solution.
Let $f(x) = x - \cos x$. Then f is continuous and

$$f(0) = -1 < 0, \qquad f\left(\frac{\pi}{2}\right) = \frac{\pi}{2} > 0.$$

The intermediate value theorem therefore gives at least one root in $(0, \pi/2)$. Trigonometric arguments must be measured in radians. $\square$

3.2 Bracketing Methods

A bracketing method begins with two points x_l and x_u such that

$$f(x_l)f(x_u) < 0.$$

Continuity then guarantees a root inside the interval. The interval is repeatedly reduced while preserving the sign change.

3.2.1 Bisection Method

Algorithm. (Bisection)

(i) Choose x_l and x_u such that $f(x_l)f(x_u) < 0$.

(ii) Compute the midpoint
$$x_r = \frac{x_l + x_u}{2}.$$

(iii) If $f(x_l)f(x_r) < 0$, set $x_u = x_r$. If $f(x_l)f(x_r) > 0$, set $x_l = x_r$. If $f(x_r) = 0$, stop.

(iv) Repeat until the required tolerance is achieved.

The absolute interval error after n bisections is bounded by

$$|r - x_r^{(n)}| \le \frac{b-a}{2^{n+1}}.$$

A commonly used estimated percentage relative error is

$$\varepsilon_a = \left| \frac{x_r^{(n)} - x_r^{(n-1)}}{x_r^{(n)}} \right| 100\%.$$

Example 34. Use bisection to solve

$$x^{10} - 1 = 0$$

with initial interval $[0, 1.3]$. Stop when the estimated percentage error is below 8%.

Solution.
Let $f(x) = x^{10} - 1$. The bisection table is

n	x_l	x_u	x_r	$\varepsilon_a(\%)$
1	0.0000	1.3000	0.6500	$--$
2	0.6500	1.3000	0.9750	33.3333
3	0.9750	1.3000	1.1375	14.2857
4	0.9750	1.1375	1.05625	7.6923

At the fourth iteration, $\varepsilon_a < 8\%$. Hence

$$r \approx 1.0563.$$

The exact positive root is 1. □

Bisection is guaranteed to converge for a continuous function with a genuine sign change. Its main disadvantages are slow linear convergence and the need for a valid bracket. It may miss roots of even multiplicity because the function does not change sign across such roots.

3.3 Open Methods

Open methods do not maintain a bracketing interval. They are generally faster when they converge, but a poor initial guess can lead to divergence or convergence to an unintended root.

3.3.1 Fixed-Point Iteration

To solve $f(x) = 0$, rearrange the equation into

$$x = g(x)$$

and iterate

$$x_{n+1} = g(x_n).$$

A root of f is then a fixed point of g.

Example 35. The function

$$f(x) = x^2 - 3x + e^x - 2$$

has a negative root. Use fixed-point iteration with

$$x_{n+1} = \frac{x_n^2 + e^{x_n} - 2}{3}, \qquad x_0 = -0.5,$$

to approximate the smaller root.

Solution.
The first few iterates are

n	x_n
0	-0.5000000000
1	-0.3811564468
2	-0.3905495820
3	-0.3902620485
4	-0.3902720192
5	-0.3902716747
6	-0.3902716861
7	-0.3902716862

Thus the smaller root is

$$r \approx -0.3902716862.$$

□

Different rearrangements of the same equation can have very different convergence behaviour. For example, a formula may be algebraically correct but have $|g'(r)| > 1$, causing nearby errors to grow rather than shrink.

Example 36. Consider

$$x_{n+1} = \frac{3 - x_n^3}{6}$$

for solving $x^3 + 6x - 3 = 0$. Compare the behaviour for $x_0 = -0.5$, 1.5, 2.5 and 5.5.

Solution.
The first three starting values eventually approach

$$r \approx 0.4814056.$$

For instance,

$$-0.5,\ 0.52083,\ 0.47645,\ 0.48197, \ldots$$

and

$$2.5,\ -2.10417,\ 2.05271,\ -0.94155,\ 0.63912, \ldots$$

both settle toward the fixed point. By contrast, $x_0 = 5.5$ produces

$$5.5,\ -27.2292,\ 3365.24,\ -6.35 \times 10^9, \ldots,$$

so the iteration diverges. Local convergence does not imply convergence from every starting value. □

Theorem 5. (Convergence of fixed-point iteration) *Suppose g is continuous on $[a, b]$ and maps $[a, b]$ into itself. Then g has at least one fixed point in $[a, b]$. If, in addition, g is differentiable and there is a constant $M < 1$ such that*

$$|g'(x)| \leq M \qquad \text{for all } x \in [a, b],$$

then the fixed point is unique and the iteration converges to it for every $x_0 \in [a, b]$.

3.3.2 Newton–Raphson Method

Newton's method replaces the graph of f by its tangent line at the current approximation. The update is

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

For a simple root and a sufficiently close starting value, convergence is usually quadratic.

Algorithm. (Newton–Raphson)

(i) Choose an initial approximation x_0.

(ii) Compute

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

(iii) Stop when $|x_{n+1} - x_n| < \varepsilon$, when $|f(x_{n+1})|$ is small, or when a maximum iteration count is reached.

Example 37. Use Newton's method to solve

$$x - e^{-x} = 0$$

for the root in $(0, 2)$. Continue until two successive approximations differ by less than 10^{-8}.

Solution.

Take $x_0 = 1$. Since

$$f(x) = x - e^{-x}, \qquad f'(x) = 1 + e^{-x},$$

we use

$$x_{n+1} = x_n - \frac{x_n - e^{-x_n}}{1 + e^{-x_n}}.$$

The iterates are

n	x_n	$\lvert x_n - x_{n-1}\rvert$
1	0.5378828427	0.4621171573
2	0.5669869914	0.0291041487
3	0.5671432860	0.0001562946
4	0.5671432904	4.42×10^{-9}

Thus

$$r \approx 0.56714329.$$

□

Newton's method can fail if $f'(x_n) = 0$, if the derivative is very small, or if the starting point lies in an unfavourable basin of attraction.

3.3.3 Secant Method

The secant method replaces the derivative by a finite-difference slope:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

It requires two initial values but no derivative. Its convergence order is approximately

$$\frac{1+\sqrt{5}}{2} \approx 1.618.$$

Example 38. Use three secant iterations to approximate $\sqrt{2}$, with $f(x) = x^2 - 2$, $x_0 = 1.5$ and $x_1 = 1$.

Solution.
The successive approximations are

$$x_2 = 1.400000, \qquad x_3 = 1.4166667, \qquad x_4 = 1.4142012.$$

Thus after three iterations,

$$\sqrt{2} \approx 1.4142.$$

□

3.4 Order of Convergence and Acceleration

Definition. (Order of convergence) Suppose $p_n \to p$ and $p_n \neq p$. If constants $\alpha > 0$ and $\lambda > 0$ exist such that

$$\lim_{n\to\infty} \frac{|p_{n+1} - p|}{|p_n - p|^\alpha} = \lambda,$$

then the sequence converges to p with order α and asymptotic error constant λ.

Convergence is called linear when $\alpha = 1$, superlinear when $1 < \alpha < 2$, and quadratic when $\alpha = 2$.

3.4.1 Aitken's Δ^2 Process

For a linearly convergent sequence $\{p_n\}$, Aitken's process forms

$$\widetilde{p}_n = p_n - \frac{(p_{n+1} - p_n)^2}{p_{n+2} - 2p_{n+1} + p_n},$$

provided the denominator is nonzero. The transformed sequence often converges substantially faster.

3.4.2 Steffensen's Method

Steffensen's method applies Aitken acceleration directly to fixed-point iteration. Starting with p_0, compute

$$p_1 = g(p_0), \qquad p_2 = g(p_1),$$

and then

$$p = p_0 - \frac{(p_1 - p_0)^2}{p_2 - 2p_1 + p_0}.$$

Replace p_0 by p and repeat.

Example 39. Use Steffensen's method to solve

$$x^3 + 4x^2 - 10 = 0, \qquad g(x) = \sqrt{\frac{10}{x+4}},$$

starting from $p_0 = 1.5$. Keep five decimal places.

Solution.
For the first cycle,

$$p_1 = g(1.5) \approx 1.34840, \qquad p_2 = g(p_1) \approx 1.36738.$$

Aitken acceleration gives

$$p \approx 1.36526.$$

Repeating the process and rounding to five decimal places gives

$$p \approx 1.36523.$$

□

3.5 Zeros of Polynomials

Theorem 6. (Fundamental theorem of algebra) *Every polynomial of degree $n \geq 1$ with real or complex coefficients has at least one complex root. Counting multiplicity, it has exactly n complex roots.*

3.5.1 Müller's Method

Müller's method fits a quadratic polynomial through three points and uses one root of that quadratic as the next approximation. It can naturally generate complex approximations even when all starting values are real.

Given x_0, x_1, x_2, write the local quadratic in the form

$$q(x) = a(x - x_2)^2 + b(x - x_2) + c, \qquad c = f(x_2).$$

The coefficients a and b satisfy

$$\begin{pmatrix} (x_0 - x_2)^2 & x_0 - x_2 \\ (x_1 - x_2)^2 & x_1 - x_2 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} f(x_0) - c \\ f(x_1) - c \end{pmatrix}.$$

Then

$$x_3 = x_2 + \frac{-2c}{b \pm \sqrt{b^2 - 4ac}},$$

where the sign is chosen to make the denominator as large as possible in magnitude.

Example 40. Use one Müller iteration for

$$f(x) = x^3 - 2x^2 - 5, \qquad x_0 = 1, \quad x_1 = 2, \quad x_2 = 3.$$

Solution.
The function values are

$$f(1) = -6, \qquad f(2) = -5, \qquad f(3) = 4.$$

Thus $c = 4$, and solving

$$\begin{pmatrix} 4 & -2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} -10 \\ -9 \end{pmatrix}$$

gives $a = 4$ and $b = 13$. Therefore

$$x_3 = 3 + \frac{-8}{13 + \sqrt{105}} \approx 2.6559.$$

□

3.6 Systems of Nonlinear Equations

Let

$$\boldsymbol{F}(\boldsymbol{X}) = \begin{pmatrix} f_1(x_1, \ldots, x_n) \\ \vdots \\ f_n(x_1, \ldots, x_n) \end{pmatrix}.$$

Newton's method for $\boldsymbol{F}(\boldsymbol{X}) = \boldsymbol{0}$ uses the Jacobian matrix

$$J(\boldsymbol{X}) = \left[\frac{\partial f_i}{\partial x_j} \right].$$

At iteration k, solve

$$J(\boldsymbol{X}^{(k)})\boldsymbol{H}^{(k)} = -\boldsymbol{F}(\boldsymbol{X}^{(k)})$$

and update

$$\boldsymbol{X}^{(k+1)} = \boldsymbol{X}^{(k)} + \boldsymbol{H}^{(k)}.$$

Example 41. Apply one Newton iteration to

$$\begin{aligned} x_1^2 - x_2^2 + 2x_2 &= 0, \\ 2x_1 + x_2^2 - 6 &= 0, \end{aligned}$$

starting from $(x_1, x_2) = (0.6, 2.2)$.

Solution.
At $\boldsymbol{X}^{(0)} = (0.6, 2.2)^\mathsf{T}$,

$$\boldsymbol{F}(\boldsymbol{X}^{(0)}) = \begin{pmatrix} -0.08 \\ 0.04 \end{pmatrix}, \qquad J(\boldsymbol{X}^{(0)}) = \begin{pmatrix} 1.2 & -2.4 \\ 2 & 4.4 \end{pmatrix}.$$

Solve

$$\begin{pmatrix} 1.2 & -2.4 \\ 2 & 4.4 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \end{pmatrix} = \begin{pmatrix} 0.08 \\ -0.04 \end{pmatrix}.$$

This gives

$$h_1 \approx 0.0254, \qquad h_2 \approx -0.0206.$$

Therefore

$$\boldsymbol{X}^{(1)} \approx \begin{pmatrix} 0.6254 \\ 2.1794 \end{pmatrix}.$$

□

Chapter 4

INTERPOLATION AND POLYNOMIAL APPROXIMATION

4.1 The Interpolation Problem

Suppose that $n+1$ data points

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

are given, where the nodes x_i are distinct. Interpolation seeks a function P satisfying

$$P(x_i) = y_i, \qquad i = 0, 1, \ldots, n.$$

In polynomial interpolation, P is chosen from

$$\mathcal{P}_n = \left\{ a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \right\}.$$

Unlike a regression curve, an interpolating polynomial passes exactly through every data point.

Theorem 7. (Weierstrass approximation theorem) *If f is continuous on $[a, b]$, then for every $\varepsilon > 0$ there exists a polynomial P such that*

$$|f(x) - P(x)| < \varepsilon \qquad \textit{for all } x \in [a, b].$$

The theorem guarantees that continuous functions can be approximated uniformly by polynomials, but it does not prescribe the degree or the best choice of nodes.

Theorem 8. (Existence and uniqueness of the interpolating polynomial) *Given $n+1$ distinct nodes $x_0, \ldots, x_n$ and arbitrary values $y_0, \ldots, y_n$, there exists exactly one polynomial $P_n \in \mathcal{P}_n$ satisfying*

$$P_n(x_i) = y_i, \qquad i = 0, \ldots, n.$$

4.2 Cardinal and Lagrange Polynomials

For each node x_k, define the cardinal polynomial

$$L_k(x) = \prod_{\substack{i=0 \\ i \neq k}}^{n} \frac{x - x_i}{x_k - x_i}.$$

These polynomials satisfy

$$L_k(x_i) = \begin{cases} 1, & i = k, \\ 0, & i \neq k. \end{cases}$$

Therefore the Lagrange interpolating polynomial is

$$P_n(x) = \sum_{k=0}^{n} y_k L_k(x).$$

At a node x_i, every term vanishes except $y_i L_i(x_i) = y_i$.

Example 42. For the data

x	1	3	4
$f(x)$	2	5	8

find the Lagrange interpolating polynomial and estimate $f(2.5)$.

Solution.
The cardinal polynomials are

$$L_0(x) = \frac{(x-3)(x-4)}{(1-3)(1-4)} = \frac{(x-3)(x-4)}{6},$$

$$L_1(x) = \frac{(x-1)(x-4)}{(3-1)(3-4)} = -\frac{(x-1)(x-4)}{2},$$

$$L_2(x) = \frac{(x-1)(x-3)}{(4-1)(4-3)} = \frac{(x-1)(x-3)}{3}.$$

Hence

$$P_2(x) = 2L_0(x) + 5L_1(x) + 8L_2(x).$$

After simplification,

$$P_2(x) = \frac{1}{2}x^2 - \frac{1}{2}x + 2.$$

Therefore

$$f(2.5) \approx P_2(2.5) = \frac{1}{2}(2.5)^2 - \frac{1}{2}(2.5) + 2 = 3.8750.$$

□

Remark 3. The Lagrange form is symmetric in the nodes and conceptually direct. However, if a new node is added, most of the basis polynomials must be rebuilt. Newton's form is usually more convenient for incremental computation.

4.3 Divided Differences and Newton's Formula

The Newton form of an interpolating polynomial is

$$P_n(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n \prod_{j=0}^{n-1}(x - x_j).$$

The coefficients are divided differences.

Definition. (Divided differences) For distinct nodes, define

$$f[x_i] = f(x_i),$$

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i},$$

and recursively

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

The Newton divided-difference polynomial is

$$\begin{aligned} P_n(x) = f[x_0] &+ f[x_0, x_1](x - x_0) \\ &+ f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots \\ &+ f[x_0, \ldots, x_n] \prod_{j=0}^{n-1} (x - x_j). \end{aligned}$$

Example 43. For the points $(1, 2)$, $(3, 5)$ and $(4, 8)$, compute $f[1]$, $f[1, 3]$ and $f[1, 3, 4]$, and construct the Newton interpolating polynomial.

Solution.
The first divided differences are

$$f[1] = 2, \qquad f[1, 3] = \frac{5 - 2}{3 - 1} = \frac{3}{2},$$

$$f[3, 4] = \frac{8 - 5}{4 - 3} = 3.$$

The second divided difference is

$$f[1, 3, 4] = \frac{f[3, 4] - f[1, 3]}{4 - 1} = \frac{3 - \frac{3}{2}}{3} = \frac{1}{2}.$$

Thus

$$P_2(x) = 2 + \frac{3}{2}(x - 1) + \frac{1}{2}(x - 1)(x - 3).$$

This simplifies to

$$P_2(x) = \frac{1}{2}x^2 - \frac{1}{2}x + 2,$$

the same unique interpolating polynomial obtained in Lagrange form. □

4.4 Forward and Backward Differences

When the nodes are equally spaced,

$$x_i = x_0 + ih,$$

divided differences can be represented efficiently by finite-difference tables.

Definition. (Forward difference) The forward difference operator Δ is defined by

$$\Delta f(x) = f(x + h) - f(x),$$

and recursively

$$\Delta^k f(x) = \Delta^{k-1} f(x + h) - \Delta^{k-1} f(x).$$

Definition. (Backward difference) The backward difference operator ∇ is defined by

$$\nabla f(x) = f(x) - f(x-h),$$

and recursively

$$\nabla^k f(x) = \nabla^{k-1} f(x) - \nabla^{k-1} f(x-h).$$

For equally spaced nodes,

$$f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k},$$

and

$$f[x_n, x_{n-1}, \ldots, x_{n-k}] = \frac{\nabla^k f(x_n)}{k! h^k}.$$

4.4.1 Newton Forward Difference Formula

Let

$$s = \frac{x - x_0}{h}.$$

Then

$$P_n(x) = f(x_0) + s\Delta f(x_0) + \frac{s(s-1)}{2!}\Delta^2 f(x_0) + \cdots + \binom{s}{n}\Delta^n f(x_0),$$

where

$$\binom{s}{k} = \frac{s(s-1)\cdots(s-k+1)}{k!}.$$

This form is most convenient when x is near the beginning of the table.

4.4.2 Newton Backward Difference Formula

Let

$$s = \frac{x - x_n}{h}.$$

Then

$$P_n(x) = f(x_n) + s\nabla f(x_n) + \frac{s(s+1)}{2!}\nabla^2 f(x_n) + \cdots + \frac{s(s+1)\cdots(s+n-1)}{n!}\nabla^n f(x_n).$$

This form is most convenient when x is near the end of the table.

Example 44. Consider the data

$$(1, 0.76519), \quad (1.3, 0.62008), \quad (1.6, 0.45541), \quad (1.9, 0.28182).$$

(a) Use Newton's forward formula to estimate $f(1.1)$.

(b) Use Newton's backward formula to estimate $f(1.8)$.

Solution.
Here $h = 0.3$. The forward-difference table begins

x	f	Δf	$\Delta^2 f$	$\Delta^3 f$
1.0	0.76519	-0.14511	-0.01956	0.01064
1.3	0.62008	-0.16467	-0.00892	
1.6	0.45541	-0.17359		
1.9	0.28182			

For $x = 1.1$,

$$s = \frac{1.1 - 1}{0.3} = \frac{1}{3}.$$

Thus

$$P(1.1) = 0.76519 + \frac{1}{3}(-0.14511) + \frac{(1/3)(-2/3)}{2}(-0.01956) + \frac{(1/3)(-2/3)(-5/3)}{6}(0.01064) \approx 0.71965.$$

For the backward formula at $x = 1.8$,

$$s = \frac{1.8 - 1.9}{0.3} = -\frac{1}{3}.$$

Using

$$\nabla f(1.9) = -0.17359, \quad \nabla^2 f(1.9) = -0.00892, \quad \nabla^3 f(1.9) = 0.01064,$$

we obtain

$$P(1.8) = 0.28182 - \frac{1}{3}(-0.17359) + \frac{(-1/3)(2/3)}{2}(-0.00892) + \frac{(-1/3)(2/3)(5/3)}{6}(0.01064) \approx 0.340018.$$

Therefore

$$f(1.1) \approx 0.71965, \qquad f(1.8) \approx 0.340018.$$

□

Remark 4. High-degree interpolation at equally spaced nodes can oscillate severely near the endpoints, a phenomenon known as Runge's phenomenon. Increasing the number of nodes does not automatically improve the approximation everywhere. Piecewise polynomials and carefully chosen nodes are often preferable.

Chapter 5

NUMERICAL DIFFERENTIATION AND INTEGRATION

5.1 Numerical Differentiation

Taylor expansions about x give

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f^{(3)}(x) + \cdots,$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2!}f''(x) - \frac{h^3}{3!}f^{(3)}(x) + \cdots.$$

By adding, subtracting or rearranging these series, derivatives can be approximated by finite differences.

Definition. (Finite-difference formulas) For a uniform step h,

$$f'(x) = \frac{f(x+h)-f(x)}{h} + \mathcal{O}(h) \qquad \text{(forward difference)},$$

$$f'(x) = \frac{f(x)-f(x-h)}{h} + \mathcal{O}(h) \qquad \text{(backward difference)},$$

$$f'(x) = \frac{f(x+h)-f(x-h)}{2h} + \mathcal{O}(h^2) \qquad \text{(central difference)},$$

and

$$f''(x) = \frac{f(x+h)-2f(x)+f(x-h)}{h^2} + \mathcal{O}(h^2).$$

The order term describes the truncation error as $h \to 0$. A central difference is normally more accurate than a one-sided difference with the same step size, provided data are available on both sides of the point.

Example 45. Given

x	1.00	1.01	1.02	1.03
$f(x)$	5.00	6.01	7.04	8.09

use $h = 0.01$ to:

(a) estimate $f'(1.02)$ by forward, backward and central differences;

(b) estimate $f''(1.02)$ by a central difference;

(c) calculate the signed percentage relative errors in part (a) if the exact derivative is 104.

Solution.
The forward estimate is

$$\frac{f(1.03) - f(1.02)}{0.01} = \frac{8.09 - 7.04}{0.01} = 105.$$

The backward estimate is

$$\frac{f(1.02) - f(1.01)}{0.01} = \frac{7.04 - 6.01}{0.01} = 103.$$

The central estimate is

$$\frac{f(1.03) - f(1.01)}{0.02} = \frac{8.09 - 6.01}{0.02} = 104.$$

For the second derivative,

$$f''(1.02) \approx \frac{8.09 - 2(7.04) + 6.01}{(0.01)^2} = 200.$$

Using

$$\frac{104 - \widehat{f'}}{104} \times 100\%,$$

the signed errors are

$$-0.9615\%, \qquad 0.9615\%, \qquad 0\%.$$

□

5.1.1 Richardson Extrapolation

Suppose an approximation has an even-power error expansion

$$D(h) = D + c_1 h^2 + c_2 h^4 + c_3 h^6 + \cdots .$$

Combining estimates at h and $h/2$ eliminates the leading h^2 term. Define

$$D_{n,0} = D\left(\frac{h}{2^n}\right).$$

The Richardson table is generated by

$$D_{n,m} = D_{n,m-1} + \frac{D_{n,m-1} - D_{n-1,m-1}}{4^m - 1}.$$

The entry $D_{n,m}$ has formal error order $\mathcal{O}(h^{2(m+1)})$.

Example 46. Let $f(x) = xe^x$. Using the central difference and Richardson extrapolation, approximate $f'(2)$ with orders $\mathcal{O}(h^2)$, $\mathcal{O}(h^4)$ and $\mathcal{O}(h^6)$, starting from $h = 0.2$.

Solution.
The central-difference values are

$$\begin{aligned} D_{0,0} &= D(0.2) = 22.414160657, \\ D_{1,0} &= D(0.1) = 22.228786880, \\ D_{2,0} &= D(0.05) = 22.182564858. \end{aligned}$$

First extrapolation gives

$$D_{1,1} = D_{1,0} + \frac{D_{1,0} - D_{0,0}}{3} = 22.166995621,$$

$$D_{2,1} = D_{2,0} + \frac{D_{2,0} - D_{1,0}}{3} = 22.167157517.$$

The next extrapolation is

$$D_{2,2} = D_{2,1} + \frac{D_{2,1} - D_{1,1}}{15} = 22.167168310.$$

Thus the requested approximations are

$$22.414161, \qquad 22.166996, \qquad 22.167169.$$

The exact derivative is $f'(2) = 3e^2 \approx 22.16716830$. □

5.2 Numerical Integration

The numerical integration problem is to approximate

$$I = \int_a^b f(x)\,\mathrm{d}x$$

when an elementary antiderivative is unavailable or when f is known only through data.

5.2.1 Composite Trapezoidal Rule

On one interval,

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{b-a}{2}\big[f(a) + f(b)\big].$$

For a uniform partition

$$x_i = a + ih, \qquad h = \frac{b-a}{n},$$

the composite trapezoidal rule is

$$T_n = \frac{h}{2}\left[f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)\right].$$

Theorem 9. **(Composite trapezoidal error bound)** *If f'' is continuous and $|f''(x)| \le M$ on $[a,b]$, then*

$$|I - T_n| \le \frac{(b-a)^3}{12n^2} M.$$

Example 47. Use the composite trapezoidal rule with $n = 4$ to estimate

$$\int_1^2 x^2\,\mathrm{d}x.$$

Solution.

Here $h = (2-1)/4 = 1/4$ and the nodes are $1, 1.25, 1.5, 1.75, 2$. Hence

$$\begin{aligned} T_4 &= \frac{1/4}{2}\left[1^2 + 2(1.25^2 + 1.5^2 + 1.75^2) + 2^2\right] \\ &= \frac{75}{32} = 2.34375. \end{aligned}$$

The exact integral is $7/3 \approx 2.33333$. □

Example 48. How many subintervals are sufficient to approximate

$$\int_1^2 \frac{1}{x}\,\mathrm{d}x$$

by the composite trapezoidal rule with error less than 10^{-4}?

Solution.
For $f(x) = 1/x$,

$$f''(x) = \frac{2}{x^3},$$

so $M = 2$ on $[1,2]$. The bound requires

$$\frac{(2-1)^3}{12n^2}(2) < 10^{-4},$$

or

$$n^2 > \frac{1}{6 \times 10^{-4}} = 1666.67.$$

Thus $n > 40.824$, and the smallest integer choice is

$$n = 41.$$

□

5.2.2 Composite Simpson Rule

On two subintervals of width $h = (b-a)/2$, Simpson's rule is

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{h}{3}\left[f(x_0) + 4f(x_1) + f(x_2)\right].$$

For an even number n of uniform subintervals,

$$S_n = \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{i=1 \\ i \text{ odd}}}^{n-1} f(x_i) + 2\sum_{\substack{i=2 \\ i \text{ even}}}^{n-2} f(x_i) + f(x_n)\right].$$

Theorem 10. (Composite Simpson error bound) *If $f^{(4)}$ is continuous and $|f^{(4)}(x)| \leq M$ on $[a,b]$, then*

$$|I - S_n| \leq \frac{(b-a)^5}{180n^4}M,$$

where n is even.

Example 49. Approximate

$$\int_0^1 4x^3 \, \mathrm{d}x$$

by composite Simpson's rule with $n = 4$.

Solution.
Simpson's rule is exact for every polynomial of degree at most three. Therefore

$$S_4 = \int_0^1 4x^3 \, \mathrm{d}x = [x^4]_0^1 = 1.$$

Direct substitution into the composite formula gives the same result. □

Example 50. Use composite Simpson's rule with $n = 10$ to approximate

$$\int_0^1 e^{x^2} \, \mathrm{d}x$$

and estimate the error using the theoretical bound.

Solution.
With $h = 0.1$, the composite Simpson sum gives

$$S_{10} \approx 1.462681.$$

For $f(x) = e^{x^2}$,

$$f^{(4)}(x) = (16x^4 + 48x^2 + 12)e^{x^2}.$$

This is increasing on $[0, 1]$, so

$$M = f^{(4)}(1) = 76e.$$

Hence

$$|E_S| \leq \frac{76e}{180(10)^4} \approx 1.15 \times 10^{-4}.$$

Thus the approximation and error bound are

$$1.462681, \qquad |E_S| \leq 0.000115.$$

□

5.2.3 Newton–Cotes Rules

Newton–Cotes formulas integrate polynomial interpolants at equally spaced nodes. Common closed rules include

$$\int_{x_0}^{x_1} f(x) \, \mathrm{d}x = \frac{h}{2}(f_0 + f_1) - \frac{h^3}{12} f''(\xi) \qquad \text{(trapezoidal)},$$

$$\int_{x_0}^{x_2} f(x) \, \mathrm{d}x = \frac{h}{3}(f_0 + 4f_1 + f_2) - \frac{h^5}{90} f^{(4)}(\xi) \qquad \text{(Simpson 1/3)},$$

$$\int_{x_0}^{x_3} f(x) \, \mathrm{d}x = \frac{3h}{8}(f_0 + 3f_1 + 3f_2 + f_3) - \frac{3h^5}{80} f^{(4)}(\xi) \qquad \text{(Simpson 3/8)}.$$

An open rule omits the endpoints. The midpoint rule, for example, is

$$\int_a^b f(x) \, \mathrm{d}x = (b - a) f\left(\frac{a + b}{2}\right) + \frac{(b - a)^3}{24} f''(\xi).$$

Open rules are useful when the integrand is undefined or poorly behaved at an endpoint.

5.3 Romberg Integration

Romberg integration applies Richardson extrapolation to a sequence of composite trapezoidal approximations. Define

$$R_{n,0} = T_{2^n}.$$

Then

$$R_{n,m} = R_{n,m-1} + \frac{R_{n,m-1} - R_{n-1,m-1}}{4^m - 1}.$$

The triangular array successively removes the $h^2, h^4, \ldots$ error terms.

Example 51. Use Romberg integration through $R_{3,3}$ to approximate

$$\int_0^1 \frac{1}{1+x^2}\,\mathrm{d}x.$$

Solution.
The Romberg table is

n	$R_{n,0}$	$R_{n,1}$	$R_{n,2}$	$R_{n,3}$
0	0.750000000			
1	0.775000000	0.783333333		
2	0.782794118	0.785392157	0.785529412	
3	0.784747124	0.785398126	0.785398524	0.785396446

Therefore

$$R_{3,3} \approx 0.785396.$$

The exact value is $\arctan(1) - \arctan(0) = \pi/4 \approx 0.785398$. □

5.4 Gaussian Quadrature

Newton–Cotes rules prescribe equally spaced nodes. Gaussian quadrature instead chooses both the nodes and weights to maximise the degree of exactness. An N-point Gauss–Legendre rule is exact for polynomials of degree at most $2N-1$.

Theorem 11. (Two-point Gauss–Legendre rule) *For a sufficiently smooth function on* $[-1, 1]$,

$$\int_{-1}^1 f(x)\,\mathrm{d}x \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

Example 52. Use two-point Gauss–Legendre quadrature to estimate

$$\int_{-1}^1 \frac{1}{x+2}\,\mathrm{d}x.$$

Solution.
The approximation is

$$G_2 = \frac{1}{2 - 1/\sqrt{3}} + \frac{1}{2 + 1/\sqrt{3}} = \frac{12}{11} \approx 1.09091.$$

The exact value is $\ln 3 \approx 1.09861$. $\square$

For an arbitrary interval $[a, b]$, use

$$x = \frac{b-a}{2}t + \frac{a+b}{2}, \qquad \mathrm{d}x = \frac{b-a}{2}\,\mathrm{d}t.$$

Thus

$$\int_a^b f(x)\,\mathrm{d}x = \frac{b-a}{2}\int_{-1}^{1} f\left(\frac{b-a}{2}t + \frac{a+b}{2}\right)\mathrm{d}t.$$

Example 53. Estimate

$$\int_{0.1}^{1.3} 5xe^{-2x}\,\mathrm{d}x$$

using two-point Gauss–Legendre quadrature.

Solution.
Here

$$x = 0.6t + 0.7, \qquad \mathrm{d}x = 0.6\,\mathrm{d}t.$$

Therefore

$$I \approx 0.6\left[f\left(0.7 - \frac{0.6}{\sqrt{3}}\right) + f\left(0.7 + \frac{0.6}{\sqrt{3}}\right)\right],$$

where $f(x) = 5xe^{-2x}$. Substitution gives

$$I \approx 0.91018.$$

$\square$

For reference, the first Gauss–Legendre rules on $[-1, 1]$ are

N	nodes	weights
2	$\pm 1/\sqrt{3}$	$1, 1$
3	$0,\ \pm\sqrt{3/5}$	$8/9,\ 5/9,\ 5/9$

Higher-order rules are usually taken from a standard table or generated computationally.

Gaussian quadrature also extends to weighted integrals

$$\int_a^b w(x)f(x)\,\mathrm{d}x.$$

Important families include Gauss–Chebyshev, Gauss–Laguerre and Gauss–Hermite rules.

Example 54. Use the two-point Gauss–Chebyshev rule to evaluate

$$\int_{-1}^{1} \frac{x^2}{\sqrt{1-x^2}}\,\mathrm{d}x.$$

Solution.
For the weight $(1-x^2)^{-1/2}$, the two nodes are $\pm 1/\sqrt{2}$ and both weights are $\pi/2$. Hence

$$I \approx \frac{\pi}{2}\left[\left(-\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{1}{\sqrt{2}}\right)^2\right] = \frac{\pi}{2}.$$

The rule is exact here because x^2 is a polynomial of sufficiently low degree. $\square$

5.5 Adaptive Quadrature

A fixed uniform mesh may waste function evaluations in smooth regions and use too few points where the integrand changes rapidly. Adaptive quadrature subdivides only those intervals whose estimated local error is too large.

For Simpson's rule on $[a, b]$, let $S^{(1)} = S(a, b)$ and let

$$S^{(2)} = S\left(a, \frac{a+b}{2}\right) + S\left(\frac{a+b}{2}, b\right).$$

Because Simpson's error scales like h^5, halving the interval suggests

$$E^{(2)} \approx \frac{1}{15}\big(S^{(2)} - S^{(1)}\big).$$

Thus the corrected approximation is

$$I \approx S^{(2)} + \frac{S^{(2)} - S^{(1)}}{15} = \frac{16S^{(2)} - S^{(1)}}{15}.$$

If

$$\frac{|S^{(2)} - S^{(1)}|}{15} < \varepsilon,$$

the interval is accepted. Otherwise it is divided into two halves, each with tolerance $\varepsilon/2$, and the test is repeated recursively.

Chapter 6

THE SOLUTION OF DIFFERENTIAL EQUATIONS

6.1 Initial-Value Problems

Definition. (Ordinary differential equation) An ordinary differential equation (ODE) is an equation involving an unknown function of one independent variable and one or more of its derivatives.

An initial-value problem (IVP) specifies the value of the solution, and possibly some derivatives, at one point. A boundary-value problem specifies conditions at different points.

Definition. (First-order IVP) A solution of

$$y' = f(x, y), \qquad y(x_0) = y_0,$$

on an interval $[a, b]$ is a differentiable function $y = \phi(x)$ such that

$$\phi(x_0) = y_0, \qquad \phi'(x) = f(x, \phi(x))$$

for every $x \in [a, b]$.

Definition. (Lipschitz condition) A function $f(x, y)$ satisfies a Lipschitz condition in y on a set D if a constant $L > 0$ exists such that

$$|f(x, y_1) - f(x, y_2)| \leq L|y_1 - y_2|$$

whenever $(x, y_1), (x, y_2) \in D$.

Theorem 12. (Existence and uniqueness) *Suppose f is continuous on*

$$D = \{(x, y) : a \leq x \leq b,\ -\infty < y < \infty\}$$

and satisfies a Lipschitz condition in y. Then the IVP

$$y' = f(x, y), \qquad y(a) = y_0,$$

has a unique solution on $[a, b]$.

A convenient sufficient condition is a bounded partial derivative: if D is convex and

$$\left|\frac{\partial f}{\partial y}(x, y)\right| \leq L$$

on D, then f is Lipschitz in y with constant L.

Example 55. Show that the IVP

$$y' = 1 + x\sin(xy), \qquad y(0) = 0, \qquad 0 \le x \le 2,$$

has a unique solution.

Solution.
The function

$$f(x, y) = 1 + x\sin(xy)$$

is continuous. Also

$$\frac{\partial f}{\partial y} = x^2 \cos(xy),$$

so on $0 \le x \le 2$,

$$\left|\frac{\partial f}{\partial y}\right| \le x^2 \le 4.$$

Thus f satisfies a Lipschitz condition in y with $L = 4$. The existence-and-uniqueness theorem applies. □

A *one-step method* uses only information from the current point (x_n, y_n) to find y_{n+1}. A *multistep method* uses data from several previous points.

6.2 Euler's Method

For mesh points

$$x_n = x_0 + nh,$$

Euler's method is

$$y_{n+1} = y_n + hf(x_n, y_n).$$

It follows from truncating the Taylor expansion

$$y(x_n + h) = y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(\xi_n).$$

The local truncation error is $\mathcal{O}(h^2)$ and the accumulated global error is $\mathcal{O}(h)$.

Example 56. Consider

$$y' = x + y, \qquad y(0) = 1.$$

(a) Use Euler's method with $h = 0.1$ to approximate $y(0.5)$.

(b) Estimate the truncation error in the first step using the next two Taylor terms.

(c) If the exact values are $y(0.1) = 1.11034$ and $y(0.5) = 1.79744$, find the errors at those points.

Solution.
The Euler recurrence is

$$y_{n+1} = y_n + 0.1(x_n + y_n).$$

It gives

n	x_n	y_n
0	0.0	1.00000
1	0.1	1.10000
2	0.2	1.22000
3	0.3	1.36200
4	0.4	1.52820
5	0.5	1.72102

Hence $y(0.5) \approx 1.72102$.

Since $y' = x + y$,

$$y'' = 1 + y' = 1 + x + y, \qquad y''' = 1 + y' = 1 + x + y.$$

At $x = 0$, $y = 1$, so $y''(0) = 2$ and $y'''(0) = 2$. The next two Taylor terms are

$$\frac{h^2}{2}y''(0) + \frac{h^3}{6}y'''(0) = \frac{0.1^2}{2}(2) + \frac{0.1^3}{6}(2) = 0.01033.$$

The actual first-step error is

$$1.11034 - 1.10000 = 0.01034.$$

At $x = 0.5$, the absolute error is

$$|1.79744 - 1.72102| = 0.07642.$$

□

Example 57. For the same IVP, compare Euler approximations using $h = 0.1$ and $h = 0.05$ with the exact solution

$$y(x) = 2e^x - x - 1.$$

Solution.
The smaller step gives consistently smaller errors:

x	$y_{h=0.1}$	$y_{h=0.05}$	$y(x)$	$E_{0.1}$	$E_{0.05}$
0.1	1.10000	1.10500	1.11034	0.01034	0.00534
0.2	1.22000	1.23101	1.24281	0.02281	0.01180
0.3	1.36200	1.38019	1.39972	0.03772	0.01953
0.4	1.52820	1.55491	1.58365	0.05545	0.02874
0.5	1.72102	1.75779	1.79744	0.07642	0.03965

Halving h approximately halves the global error, consistent with Euler's first-order accuracy.
□

6.3 Improved Euler Method

Improved Euler, also called Heun's method or the explicit trapezoidal method, uses a predictor and corrector:

$$y_{n+1}^* = y_n + hf(x_n, y_n),$$
$$y_{n+1} = y_n + \frac{h}{2}\left[f(x_n, y_n) + f(x_{n+1}, y_{n+1}^*)\right].$$

Its local error is $\mathcal{O}(h^3)$ and its global error is $\mathcal{O}(h^2)$.

Example 58. Use improved Euler with $h = 0.1$ for

$$y' = x + y, \qquad y(0) = 1,$$

on $[0, 1]$. Compare the approximations with $y(x) = 2e^x - x - 1$.

Solution.
The results are

x_k	y_k	$y(x_k)$	$\lvert y(x_k) - y_k\rvert$
0.0	1.00000	1.00000	0.00000
0.1	1.11000	1.11034	0.00034
0.2	1.24205	1.24281	0.00076
0.3	1.39847	1.39972	0.00125
0.4	1.58180	1.58365	0.00184
0.5	1.79489	1.79744	0.00254
0.6	2.04086	2.04424	0.00338
0.7	2.32315	2.32751	0.00436
0.8	2.64558	2.65108	0.00550
0.9	3.01237	3.01921	0.00684
1.0	3.42817	3.43656	0.00839

The errors are much smaller than those from Euler's method with the same step size because the method is second order globally. □

6.4 Taylor Series Methods

A Taylor method of order p uses

$$y_{n+1} = y_n + hy_n' + \frac{h^2}{2!}y_n'' + \frac{h^3}{3!}y_n^{(3)} + \cdots + \frac{h^p}{p!}y_n^{(p)}.$$

Its local truncation error is $\mathcal{O}(h^{p+1})$ and its global truncation error is $\mathcal{O}(h^p)$. The main drawback is that higher derivatives must be derived and evaluated.

Example 59. For

$$y' = 2xy, \qquad y(1) = 1,$$

use a second-order Taylor method with $h = 0.1$ to approximate $y(1.2)$.

Solution.
Since

$$y' = 2xy,$$

we have

$$y'' = 2y + 2xy' = 2y + 4x^2y = 2(1 + 2x^2)y.$$

The recurrence is

$$y_{n+1} = y_n + 0.1(2x_ny_n) + \frac{0.1^2}{2}2(1 + 2x_n^2)y_n.$$

At $(x_0, y_0) = (1, 1)$,

$$y_1 = 1 + 0.2 + 0.03 = 1.23.$$

At $(x_1, y_1) = (1.1, 1.23)$,

$$y_1' = 2.706, \qquad y_1'' = 8.4132,$$

so

$$y_2 = 1.23 + 0.1(2.706) + 0.005(8.4132) = 1.542666.$$

Therefore

$$y(1.2) \approx 1.5427.$$

□

6.5 Runge–Kutta Methods

Runge–Kutta methods achieve high order without explicitly differentiating f. The classical fourth-order method is

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

where

$$k_1 = hf(x_n, y_n),$$
$$k_2 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right),$$
$$k_3 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right),$$
$$k_4 = hf(x_n + h, y_n + k_3).$$

RK4 has local error $\mathcal{O}(h^5)$ and global error $\mathcal{O}(h^4)$.

Example 60. Use RK4 with $h = 0.1$ to approximate $y(0.2)$ for

$$y' = x + y, \qquad y(0) = 1.$$

Solution.
For the first step,

$$k_1 = 0.100000, \quad k_2 = 0.110000, \quad k_3 = 0.110500, \quad k_4 = 0.121050,$$

which gives

$$y_1 = 1.110341667.$$

For the second step,

$$k_1 = 0.121034167, \quad k_2 = 0.132085875, \quad k_3 = 0.132638460, \quad k_4 = 0.144298013.$$

Hence

$$y_2 = 1.242805142.$$

Therefore

$$y(0.2) \approx 1.2428.$$

□

6.5.1 Adaptive Runge–Kutta–Fehlberg Method

The RKF45 method computes a fourth-order estimate $\widetilde{y}_{n+1}$ and a fifth-order estimate y_{n+1} from six shared stage values. Their difference estimates the local error:

$$\varepsilon_c = |y_{n+1} - \widetilde{y}_{n+1}|.$$

If the current step is h_c and the desired local tolerance is ε_0, a typical new step is

$$h_0 = h_c \left(\frac{\varepsilon_0}{\varepsilon_c}\right)^{1/5}.$$

If $\varepsilon_c \le \varepsilon_0$, the step is accepted and the next step size is adjusted. Otherwise the result is rejected and recomputed with the smaller step.

6.6 Multistep Methods

Multistep methods use several previous function values. Explicit Adams–Bashforth formulas predict a new value, while implicit Adams–Moulton formulas involve the unknown new value and are commonly used as correctors.

The four-step Adams–Bashforth formula is

$$y_{n+1} = y_n + \frac{h}{24}\left(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}\right),$$

where $f_j = f(x_j, y_j)$.

The three-step Adams–Moulton corrector of matching fourth order is

$$y_{n+1} = y_n + \frac{h}{24}\left(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}\right).$$

A predictor–corrector implementation uses

$$y^*_{n+1} = y_n + \frac{h}{24}\left(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}\right)$$

and then

$$y_{n+1} = y_n + \frac{h}{24}\left(9f(x_{n+1}, y^*_{n+1}) + 19f_n - 5f_{n-1} + f_{n-2}\right).$$

The method is not self-starting; values y_1, y_2, y_3 are normally generated with RK4.

Example 61. Use the fourth-order Adams–Bashforth/Adams–Moulton method with $h = 0.2$ to approximate $y(0.8)$ for

$$y' = x + y, \qquad y(0) = 1.$$

Solution.
Using RK4 for the starting values gives

$$y_0 = 1, \quad y_1 = 1.242800, \quad y_2 = 1.583636, \quad y_3 = 2.044213.$$

The corresponding function values are

$$f_0 = 1, \quad f_1 = 1.442800, \quad f_2 = 1.983636, \quad f_3 = 2.644213.$$

The Adams–Bashforth predictor is

$$y_4^* = y_3 + \frac{0.2}{24}(55f_3 - 59f_2 + 37f_1 - 9f_0) = 2.650720.$$

Then

$$f_4^* = 0.8 + y_4^* = 3.450720,$$

and the Adams–Moulton corrector gives

$$y_4 = y_3 + \frac{0.2}{24}(9f_4^* + 19f_3 - 5f_2 + f_1) = 2.651056.$$

Therefore

$$y(0.8) \approx 2.6510.$$

□

6.7 Higher-Order Initial-Value Problems

A higher-order ODE can be rewritten as a first-order system. For example,

$$y'' = f(x, y, y'), \qquad y(x_0) = y_0, \qquad y'(x_0) = u_0,$$

becomes

$$y' = u, \qquad u' = f(x, y, u).$$

Any first-order numerical method can then be applied componentwise.

6.7.1 Euler Method for a System

For

$$y' = g(x, y, u), \qquad u' = f(x, y, u),$$

Euler's method is

$$y_{n+1} = y_n + hg(x_n, y_n, u_n),$$
$$u_{n+1} = u_n + hf(x_n, y_n, u_n).$$

Example 62. Use Euler's method in two steps to approximate $y(0.2)$ and $y'(0.2)$ for

$$y'' + xy' + y = 0, \qquad y(0) = 1, \qquad y'(0) = 2.$$

Solution.

Let $u = y'$. Then

$$y' = u, \qquad u' = -xu - y.$$

With $h = 0.1$ and $(y_0, u_0) = (1, 2)$,

$$y_1 = 1 + 0.1(2) = 1.2, \qquad u_1 = 2 + 0.1[-0(2) - 1] = 1.9.$$

At $x_1 = 0.1$,

$$y_2 = 1.2 + 0.1(1.9) = 1.39,$$
$$u_2 = 1.9 + 0.1[-0.1(1.9) - 1.2] = 1.761.$$

Thus

$$y(0.2) \approx 1.39, \qquad y'(0.2) \approx 1.761.$$

□

6.7.2 RK4 for a System

For

$$y' = p(x, y, u), \qquad u' = q(x, y, u),$$

RK4 computes paired stages (k_i, m_i):

$$k_1 = hp(x_n, y_n, u_n), \qquad m_1 = hq(x_n, y_n, u_n),$$
$$k_2 = hp\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}, u_n + \frac{m_1}{2}\right),$$
$$m_2 = hq\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}, u_n + \frac{m_1}{2}\right),$$

with analogous formulas for (k_3, m_3) and (k_4, m_4). Then

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$
$$u_{n+1} = u_n + \frac{1}{6}(m_1 + 2m_2 + 2m_3 + m_4).$$

Example 63. For

$$y'' = 4(y - x), \qquad y(0) = 0, \qquad y'(0) = 0,$$

use one RK4 step with $h = 1/3$ to approximate $y(1/3)$ and $y'(1/3)$.

Solution.
Let $u = y'$. The system is

$$y' = u, \qquad u' = 4(y - x).$$

Applying the paired RK4 stages over $h = 1/3$ gives

$$y_1 \approx -0.02469, \qquad u_1 \approx -0.23045.$$

Therefore

$$y\left(\frac{1}{3}\right) \approx -0.0247, \qquad y'\left(\frac{1}{3}\right) \approx -0.2305.$$

□

6.8 Stability and Stiffness

A numerical method is stable if local perturbations and round-off errors do not grow catastrophically as the computation proceeds.

For the test equation

$$y' = -\lambda y, \qquad \lambda > 0,$$

Euler's method gives

$$y_{n+1} = (1 - \lambda h) y_n.$$

Stability requires

$$|1 - \lambda h| \le 1,$$

so

$$0 \le h \le \frac{2}{\lambda}.$$

For a system

$$\boldsymbol{y}' = -A\boldsymbol{y}$$

with positive eigenvalues, Euler stability requires approximately

$$h \le \frac{2}{\lambda_{\max}}.$$

An IVP is stiff when some solution components decay or vary much more rapidly than others. For

$$\boldsymbol{y}' = -A\boldsymbol{y},$$

widely separated eigenvalue magnitudes indicate stiffness. An explicit method may then require an extremely small step for stability even when the slowly varying part of the solution would not require it for accuracy.

Example 64. For

$$y'' + 1001y' + 1000y = 0,$$

find a step-size restriction for Euler's method to be stable.

Solution.
The characteristic polynomial is

$$r^2 + 1001r + 1000 = (r + 1)(r + 1000),$$

so the decay rates are 1 and 1000. The largest eigenvalue magnitude governing Euler stability is 1000. Therefore

$$h \le \frac{2}{1000} = 0.002.$$

The very different rates 1 and 1000 also show that the problem is stiff. □

Chapter 7

BOUNDARY-VALUE PROBLEMS FOR ODES

7.1 Two-Point Boundary-Value Problems

In a second-order initial-value problem, both y and y' are specified at the same point. In a two-point boundary-value problem (BVP), values of y are specified at different endpoints.

Definition. (Two-point BVP) A two-point boundary-value problem has the form

$$y'' = f(x, y, y'), \qquad a \le x \le b,$$

subject to

$$y(a) = \alpha, \qquad y(b) = \beta.$$

Definition. (Linear second-order equation) The equation is linear if it can be written as

$$y'' = p(x)y' + q(x)y + r(x),$$

or equivalently

$$y'' - p(x)y' - q(x)y = r(x).$$

The two principal numerical approaches in this topic are the shooting method and the finite-difference method.

7.2 Linear Shooting Method

Consider

$$y'' = p(x)y' + q(x)y + r(x), \qquad y(a) = \alpha, \qquad y(b) = \beta.$$

The linear shooting method replaces the BVP by two IVPs.

First solve

$$y_1'' = p(x)y_1' + q(x)y_1 + r(x), \qquad y_1(a) = \alpha, \qquad y_1'(a) = 0.$$

Next solve the homogeneous IVP

$$y_2'' = p(x)y_2' + q(x)y_2, \qquad y_2(a) = 0, \qquad y_2'(a) = 1.$$

Because of linearity, every solution satisfying the left boundary condition has the form

$$y(x) = y_1(x) + cy_2(x).$$

The right boundary condition gives

$$c = \frac{\beta - y_1(b)}{y_2(b)},$$

provided $y_2(b) \neq 0$. Therefore

$$\boxed{y(x) = y_1(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2(x)}.$$

The two IVPs may be integrated by RK4 or another IVP solver.

Example 65. Apply linear shooting with RK4 and $h = 1/3$ to

$$y'' = 4(y - x), \qquad 0 \le x \le 1, \qquad y(0) = 0, \qquad y(1) = 2.$$

Estimate $y(1/3)$ and $y(2/3)$.

Solution.
The first IVP is

$$y_1'' = 4(y_1 - x), \qquad y_1(0) = 0, \qquad y_1'(0) = 0.$$

RK4 gives

x	y_1	y_1'
0	0	0
1/3	−0.024691	−0.230453
2/3	−0.214398	−1.026741
1	−0.809732	−2.755580

The homogeneous IVP is

$$y_2'' = 4y_2, \qquad y_2(0) = 0, \qquad y_2'(0) = 1.$$

RK4 gives

x	y_2	y_2'
0	0	1
1/3	0.358025	1.230453
2/3	0.881065	2.026741
1	1.809732	3.755580

The shooting coefficient is

$$c = \frac{2 - y_1(1)}{y_2(1)} = \frac{2.809732}{1.809732} \approx 1.552568.$$

Therefore

$$y\left(\frac{1}{3}\right) \approx -0.024691 + 1.552568(0.358025) = 0.5312,$$
$$y\left(\frac{2}{3}\right) \approx -0.214398 + 1.552568(0.881065) = 1.1535.$$

□

7.3 Shooting for Nonlinear Problems

For

$$y'' = f(x, y, y'), \qquad y(a) = \alpha, \qquad y(b) = \beta,$$

treat the unknown initial slope as a parameter:

$$y'(a) = z.$$

For a chosen z, solve the IVP and denote the computed endpoint value by

$$y(b) = \phi(z).$$

The correct slope solves the scalar nonlinear equation

$$F(z) = \phi(z) - \beta = 0.$$

Thus a nonlinear shooting method combines an IVP solver with a root-finding method such as bisection, secant or Newton iteration. Each root iteration requires one or more integrations of the IVP.

Algorithm. (Nonlinear shooting)

(i) Choose a trial slope z.

(ii) Solve

$$y'' = f(x, y, y'), \qquad y(a) = \alpha, \qquad y'(a) = z.$$

(iii) Compute the boundary residual $F(z) = y(b; z) - \beta$.

(iv) Adjust z with a root-finding method and repeat until $|F(z)|$ is sufficiently small.

7.4 Finite-Difference Method

Consider the linear BVP

$$y'' + P(x)y' + Q(x)y = f(x), \qquad y(a) = \alpha, \qquad y(b) = \beta.$$

Let

$$x_i = a + ih, \qquad h = \frac{b-a}{n}, \qquad y_i \approx y(x_i).$$

At each interior node $i = 1, \ldots, n-1$, use central differences:

$$y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2},$$

$$y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h}.$$

Substitution gives

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + P_i \frac{y_{i+1} - y_{i-1}}{2h} + Q_i y_i = f_i.$$

After multiplying by h^2,

$$\left(1 - \frac{h}{2}P_i\right) y_{i-1} + (h^2 Q_i - 2)y_i + \left(1 + \frac{h}{2}P_i\right) y_{i+1} = h^2 f_i.$$

The boundary values $y_0 = \alpha$ and $y_n = \beta$ are moved to the right-hand side. The unknown interior values satisfy a tridiagonal linear system.

Example 66. Use the finite-difference method with $h = 1$ to approximate the solution of

$$y'' - \left(1 - \frac{x}{5}\right) y = x, \qquad y(1) = 2, \qquad y(3) = -1.$$

Find $y(2)$.

Solution.
There is one interior point, $x_1 = 2$, with unknown $y_1 \approx y(2)$. Here

$$P(x) = 0, \qquad Q(x) = -\left(1 - \frac{x}{5}\right), \qquad f(x) = x.$$

At $x = 2$,

$$Q_1 = -\frac{3}{5}, \qquad f_1 = 2.$$

The finite-difference equation is

$$y_0 + (Q_1 - 2)y_1 + y_2 = f_1,$$

because $h = 1$. Substituting $y_0 = 2$ and $y_2 = -1$ gives

$$2 - \frac{13}{5}y_1 - 1 = 2.$$

Thus

$$-\frac{13}{5}y_1 = 1, \qquad y_1 = -\frac{5}{13} \approx -0.3846.$$

Therefore

$$y(2) \approx -0.3846.$$

□

Remark 5. Shooting preserves the differential-equation viewpoint but can be sensitive when the endpoint value changes greatly with the initial slope. Finite differences convert the entire BVP directly into a sparse linear or nonlinear algebraic system and are often more robust for difficult boundary-value problems.

Chapter 8

NUMERICAL SOLUTIONS TO PARTIAL DIFFERENTIAL EQUATIONS

8.1 Classification of Second-Order PDEs

Definition. (Linear second-order PDE) A linear second-order PDE in two variables has the form

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu = G,$$

where the coefficients may depend on the independent variables. It is classified by the discriminant $B^2 - 4AC$:

(a) parabolic if $B^2 - 4AC = 0$;

(b) hyperbolic if $B^2 - 4AC > 0$;

(c) elliptic if $B^2 - 4AC < 0$.

Example 67. Classify the one-dimensional wave equation, the one-dimensional heat equation and the two-dimensional Laplace equation.

Solution.
For the wave equation

$$u_{tt} = c^2 u_{xx},$$

write $c^2 u_{xx} - u_{tt} = 0$. Then $A = c^2$, $B = 0$, $C = -1$, so

$$B^2 - 4AC = 4c^2 > 0.$$

It is hyperbolic.

For the heat equation

$$u_t = c^2 u_{xx},$$

the coefficient of the second derivative in time is zero, giving discriminant 0. It is parabolic.

For Laplace's equation

$$u_{xx} + u_{yy} = 0,$$

$A = C = 1$ and $B = 0$, so

$$B^2 - 4AC = -4 < 0.$$

It is elliptic. □

8.2 Finite Differences for Two-Variable Functions

Let

$$x_i = x_0 + ih, \qquad t_j = t_0 + jk, \qquad u_{i,j} \approx u(x_i, t_j).$$

Typical formulas are

$$\begin{aligned}
u_x(x_i,t_j) &= \frac{u_{i+1,j} - u_{i,j}}{h} + \mathcal{O}(h), \\
u_t(x_i,t_j) &= \frac{u_{i,j+1} - u_{i,j}}{k} + \mathcal{O}(k), \\
u_x(x_i,t_j) &= \frac{u_{i+1,j} - u_{i-1,j}}{2h} + \mathcal{O}(h^2), \\
u_{xx}(x_i,t_j) &= \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \mathcal{O}(h^2).
\end{aligned}$$

A finite-difference scheme replaces every derivative in the PDE by such algebraic expressions and then solves for the grid values.

8.3 Parabolic Problems: The Heat Equation

Consider

$$u_t = u_{xx}, \qquad 0 < x < a, \qquad t > 0,$$

with an initial temperature distribution and boundary values.

8.3.1 FTCS Explicit Scheme

Using a forward difference in time and a central difference in space gives

$$\frac{u_{i,j+1} - u_{i,j}}{k} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}.$$

Define

$$r = \frac{k}{h^2}.$$

Then

$$\boxed{u_{i,j+1} = (1-2r)u_{i,j} + r(u_{i+1,j} + u_{i-1,j})}.$$

The method is explicit because all values at level $j+1$ are computed directly from the known level j.

Example 68. For

$$u_t = u_{xx}, \qquad 0 \le x \le 1,$$

with

$$u(x,0) = \sin(\pi x), \qquad u(0,t) = u(1,t) = 0,$$

use FTCS with $h = 0.1$ and $k = 0.0005$ to estimate $u(0.1, 0.001)$.

Solution.
Here

$$r = \frac{0.0005}{0.1^2} = 0.05.$$

At time $t = 0$,

$$u_{1,0} = \sin(0.1\pi) = 0.309017, \quad u_{2,0} = \sin(0.2\pi) = 0.587785, \quad u_{0,0} = 0.$$

The first time step gives

$$u_{1,1} = 0.9u_{1,0} + 0.05(u_{2,0} + u_{0,0}) = 0.307505.$$

After computing the neighbouring values at $t = 0.0005$, the second step gives

$$u_{1,2} = 0.3059995.$$

Therefore

$$u(0.1, 0.001) \approx 0.3060.$$

□

Theorem 13. (FTCS stability condition) *For the one-dimensional heat equation, the FTCS scheme is stable only when*

$$0 < r = \frac{k}{h^2} \leq \frac{1}{2}.$$

Thus refining the spatial grid forces a much smaller time step. If h is halved, the stability limit requires k to be reduced by a factor of four.

8.3.2 Crank–Nicolson Implicit Method

Crank–Nicolson averages the spatial central differences at time levels j and $j + 1$:

$$\frac{u_{i,j+1} - u_{i,j}}{k} = \frac{1}{2}\left[\frac{u_{i+1,j+1} - 2u_{i,j+1} + u_{i-1,j+1}}{h^2} + \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}\right].$$

With $r = k/h^2$, rearrangement gives

$$-ru_{i-1,j+1} + 2(1 + r)u_{i,j+1} - ru_{i+1,j+1} = ru_{i-1,j} + 2(1 - r)u_{i,j} + ru_{i+1,j}.$$

At every new time level, the interior values are obtained from a tridiagonal linear system. Crank–Nicolson is second-order accurate in both space and time and has no analogous restriction $r \leq 1/2$ for stability.

Example 69. For the same heat problem, use Crank–Nicolson with $h = 0.2$ and $k = 0.001$ to estimate $u(0.4, 0.001)$.

Solution.
Here

$$r = \frac{0.001}{0.2^2} = 0.025.$$

There are four interior unknowns at the new time level. They satisfy

$$\begin{pmatrix} 2.05 & -0.025 & 0 & 0 \\ -0.025 & 2.05 & -0.025 & 0 \\ 0 & -0.025 & 2.05 & -0.025 \\ 0 & 0 & -0.025 & 2.05 \end{pmatrix} \begin{pmatrix} u_{1,1} \\ u_{2,1} \\ u_{3,1} \\ u_{4,1} \end{pmatrix} = \begin{pmatrix} 1.169958 \\ 1.893031 \\ 1.893031 \\ 1.169958 \end{pmatrix}.$$

Solving gives

$$(u_{1,1}, u_{2,1}, u_{3,1}, u_{4,1}) \approx (0.582199, 0.942018, 0.942018, 0.582199).$$

Since $x = 0.4$ corresponds to $i = 2$,

$$u(0.4, 0.001) \approx 0.9420.$$

□

For this problem, the exact solution is

$$u(x,t) = e^{-\pi^2 t} \sin(\pi x).$$

This allows explicit and implicit schemes to be compared with the true solution. FTCS can be very accurate when its stability condition is respected, but it may become unstable when k/h^2 is too large. Crank–Nicolson remains stable and usually gives better accuracy at larger time steps.

8.4 Elliptic Problems: Laplace's Equation

For

$$u_{xx} + u_{yy} = 0,$$

use central differences on a rectangular grid with spacings h and k:

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} = 0.$$

For a square grid $h = k$, this simplifies to the five-point formula

$$\boxed{u_{i,j} = \frac{1}{4}\left(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}\right)}.$$

Thus each interior temperature is the average of its four nearest neighbours. All interior equations form a sparse linear system, commonly solved by Jacobi, Gauss–Seidel or SOR.

Example 70. A square plate of side 12 cm has temperatures

$$\text{left} = extright = extbottom = 100°\text{C}, \qquad \text{top} = 0°\text{C}.$$

Using a mesh size of 4 cm, find the temperatures at

$$A(4,4), \quad B(8,4), \quad C(8,8), \quad D(4,8).$$

Gauss–Seidel starts from

$$u_A = u_B = 80, \qquad u_C = u_D = 50,$$

and stops when every change is below 10^{-3}.

Solution.
The five-point equations are

$$u_A = \frac{100 + u_B + u_D + 100}{4},$$

$$u_B = \frac{u_A + 100 + u_C + 100}{4},$$
$$u_C = \frac{u_D + 100 + 0 + u_B}{4},$$
$$u_D = \frac{100 + u_C + 0 + u_A}{4}.$$

By symmetry, $u_A = u_B$ and $u_C = u_D$. Let these common values be a and c. Then

$$a = 50 + \frac{a+c}{4}, \qquad c = 25 + \frac{a+c}{4}.$$

Equivalently,

$$3a - c = 200, \qquad -a + 3c = 100.$$

Solving gives

$$a = 87.5, \qquad c = 62.5.$$

Gauss–Seidel converges to the same values:

$$u_A = u_B = 87.5°\text{C}, \qquad u_C = u_D = 62.5°\text{C}.$$

□

8.5 Hyperbolic Problems: The Wave Equation

Consider

$$u_{tt} = u_{xx},$$

with fixed-end boundary conditions and initial displacement and velocity:

$$u(0,t) = u(a,t) = 0,$$

$$u(x,0) = f(x), \qquad u_t(x,0) = g(x).$$

Using central differences in both time and space gives

$$\frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}.$$

Let

$$\rho = \left(\frac{k}{h}\right)^2.$$

Then the CTCS scheme is

$$\boxed{u_{i,j+1} = \rho u_{i+1,j} + 2(1-\rho)u_{i,j} + \rho u_{i-1,j} - u_{i,j-1}}.$$

The stability condition is

$$0 \le \rho \le 1, \qquad \text{equivalently } k \le h.$$

The initial velocity requires a special first time level. A central approximation at $t = 0$ gives

$$\frac{u_{i,1} - u_{i,-1}}{2k} = g_i.$$

Eliminating the fictitious value $u_{i,-1}$ from the CTCS equation yields

$$u_{i,1} = \frac{\rho}{2}(f_{i+1} + f_{i-1}) + (1-\rho)f_i + kg_i.$$

Example 71. A string with fixed ends satisfies

$$u_{tt} = u_{xx}, \qquad 0 \le x \le 1,$$

with

$$u(x,0) = 0, \qquad u_t(x,0) = \sin(\pi x),$$

and $u(0,t) = u(1,t) = 0$. Use CTCS with $h = k = 0.2$ to find the displacement at $t = 0.4$ for $x = 0.2, 0.4, 0.6, 0.8$.

Solution.
Since $h = k$, we have $\rho = 1$. The initial displacement is $f_i = 0$, so the first time level is

$$u_{i,1} = kg_i = 0.2\sin(\pi x_i).$$

Thus at $t = 0.2$,

$$(u_{1,1}, u_{2,1}, u_{3,1}, u_{4,1}) \approx (0.11756, 0.19021, 0.19021, 0.11756).$$

For the next level, because $\rho = 1$ and $u_{i,0} = 0$,

$$u_{i,2} = u_{i+1,1} + u_{i-1,1}.$$

Using the fixed boundary values $u_{0,1} = u_{5,1} = 0$, we obtain

$$\begin{aligned}
u_{1,2} &= 0.19021, \\
u_{2,2} &= 0.11756 + 0.19021 = 0.30777, \\
u_{3,2} &= 0.19021 + 0.11756 = 0.30777, \\
u_{4,2} &= 0.19021.
\end{aligned}$$

Therefore at $t = 0.4$ the displacements are approximately

$$0.1902, \quad 0.3078, \quad 0.3078, \quad 0.1902.$$

□